

RÉPUBLIQUE DU NIGER

LE MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR, DE LA RECHERCHE ET DE L'INNOVATION
TECHNOLOGIQUE

ÉCOLE DES MINES DE L'INDUSTRIE ET DE LA GÉOLOGIE (EMIG)



Etablissement public à caractère scientifique, culturel et technique.

Créé par la loi n° 2004 – 019 du 16 mai 2004

BP. : 732 Niamey – Niger

Courriel. : emig@intnet.ne; Site web. : www.emig-niger.org



RAPPORT DE STAGE DE FIN DE CYCLE LICENCE

Département : GÉNIE INFORMATIQUE

Cycle : LICENCE

Option : INFORMATIQUE – GÉNIE LOGICIEL

THÈME :

**CONCEPTION ET RÉALISATION D'UNE PLATEFORME DE GESTION DU SERVICE RESTAURANT
DE L'EMIG AVEC UNE EXTENSION MOBILE DÉDIÉE A LA RESERVATION DES PLATS PAR LES
ÉLÈVES**



NOVATECH Niger est une entreprise Numérique spécialisée dans les Services Réseaux, Serveurs Cloud, Développement Web et Mobile, Formations & Certifications, Énergie, Électronique et Consulting. Niamey, Niger

Présenté par :

M. Moussa Ismael

Mlle. Mahaman Lawan Abou Tasnim

Tuteur en entreprise :

Mr Daouda Hamadou

Responsable pédagogique :

Dr. Alkassoum Nabil

PROMOTION 2023

JUIN 2025

Remerciements

Nous tenons à exprimer notre sincère reconnaissance à notre responsable pédagogique, Dr. Alkassoum Nabil pour son accompagnement précieux, ses conseils avisés et la confiance qu'il nous a accordée tout au long de ce stage. Sa guidance éclairée et son soutien constant ont été déterminants dans la réalisation de ce projet et l'approfondissement de nos connaissances.

Nos remerciements les plus chaleureux s'adressent également à l'ensemble du personnel de Novatech, et plus particulièrement à Mr Hamidou Daouda pour son accueil exceptionnel, sa disponibilité et sa volonté de partager son expertise. Cette opportunité d'immersion au sein de leurs équipes a été une expérience enrichissante et formatrice, nous permettant d'acquérir une compréhension concrète des enjeux professionnels.

Nous sommes également reconnaissants envers tous les professeurs du *Département Génie Informatique de l'EMIG* dont la qualité de l'enseignement et le dévouement ont été des piliers essentiels de notre formation et nous ont préparés efficacement à cette expérience en entreprise.

Un merci spécial à nos camarades de promotion et amis, pour les échanges stimulants, le soutien mutuel et les moments de convivialité qui ont enrichi notre parcours. Leur présence a été une source de motivation inestimable.

Enfin, nous souhaitons témoigner toute notre gratitude à nos familles et à nos proches pour leur soutien indéfectible, leur patience et leur compréhension. Leur présence à nos côtés a constitué une source de force et d'inspiration continue.

À toutes les personnes ayant contribué à cette expérience, nous adressons nos plus vifs remerciements pour leur aide précieuse et leur appui sans faille. Ce projet n'aurait pu aboutir sans leur généreuse contribution.

Table des matières

Remerciements	I
Sigles et abréviations	IV
Liste des figures.....	VI
Introduction générale	1
1. Présentation de l'entreprise d'accueil : Novatech.....	2
1.1 Historique et contexte de création	2
1.2 Vision, missions et valeurs	2
1.3 Organisation et fonctionnement.....	3
1.4 Domaines d'expertise.....	3
1.5 Conclusion du Chapitre 1	4
2. Présentation générale de l'EMIG	5
2.1 Historique et statut de l'EMIG.....	5
2.2 Fonctionnement et organisation de l'EMIG	5
2.3 La formation initiale	6
2.4 Présentation du Département Génie Informatique	7
2.5 Description de la Direction des Moyens Généraux et de la Cité et du Service Restaurant.....	7
2.5.1 Description de la Direction des Moyens Généraux et de la Cité.....	7
2.6.2 Description du Service Restaurant	7
3. Concepts de Base	9
3.1 Introduction	9
3.2 Définition des applications web	9
3.3 Définition des applications mobiles	9
3.4 Les systèmes d'exploitation mobiles (Android, iOS)	10
3.5 Avantages des applications mobiles dans un contexte universitaire	10
3.6 Comparaison entre les applications mobiles et web	11
3.7 Conclusion du Chapitre 3	12
4. Analyse et Spécification des Besoins.....	13
4.1 Introduction	13
4.2 Étude de l'existant	13
4.2.1 Organisation actuelle du service restaurant	13
4.3 Critique de l'existant.....	14
4.3.1 Limites et contraintes observées	14
4.4 Solution proposée	15
4.5 Acteurs du système et leurs rôles	16
4.6 Besoins fonctionnels	17

4.7 Besoins non fonctionnels	17
4.8 Contraintes techniques	17
4.9 Présentation du projet global	18
4.9.1. Contexte du développement	19
4.9.2. Périmètre du travail réalisé durant le stage	19
4.10 Objectifs généraux et spécifiques	19
4.10.1 Objectifs spécifiques	19
4.10.2 Démarche méthodologique adoptée	20
4.11 Conclusion du Chapitre 4	22
5. Conception	23
5.1 Introduction	23
5.2 Modélisation UML.....	23
5.2.1 Diagramme de cas d'utilisation	24
5.2.2 Diagramme de classes.....	26
5.2.3 Diagramme de séquence	29
5.3 Conclusion du Chapitre 5	33
6. Réalisation de l'Application	33
6.1 Introduction	34
6.2 Choix de l'environnement de développement.....	34
6.2.1 Architecture de l'application	35
6.2.1.1 Backend (Serveur)	35
6.2.1.2 Authentification et sécurité	35
6.2.1.3 Endpoints principaux	35
6.2.1.4 Frontend Web (interface administrateur).....	36
6.2.1.5 Application mobile (extension pour les étudiants)	36
6.2.2 Schéma de la base de données.....	36
6.2.3 Interface de l'application mobile (étudiants)	37
6.2.4 Connexion entre l'application mobile et le serveur	41
6.2.5 Tests et validation.....	42
6.3 Conclusion du Chapitre 6	43
Conclusion Générale	44

Sigles et abréviations

Sigle / Abréviation	Définition
API	Application Programming Interface (Interface de Programmation Applicative)
APK	Android Package Kit
CEDEAO	Communauté Économique des États de l'Afrique de l'Ouest
CEAO	Communauté Économique de l'Afrique de l'Ouest
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer)
CSS	Cascading Style Sheets (Feuilles de Style en Cascade)
DB	DataBase (Base de Données)
DMGC	Directeur des Moyens Généraux de la Cité
DRF	Django REST Framework
E2E	End to End (Tests de bout en bout)
ECUE	Élément Constitutif d'Unité d'Enseignement
EMIG	École des Mines de l'Industrie et de la Géologie
EPSCT	Établissement Public à Caractère Scientifique, Culturel et Technique
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMD	Licence-Master-Doctorat
OMG	Object Management Group
OTP	One Time Password (Mot de passe à usage unique)
PWA	Progressive Web App

Sigle / Abréviation	Définition
QR Code	Quick Response Code
REST	Representational State Transfer
SPA	Single Page Application (Application Monopage)
SQL	Structured Query Language
TIC	Technologies de l'Information et de la Communication
ToIP	Telephony over IP
UE	Unité d'Enseignement
UML	Unified Modeling Language
URL	Uniform Resource Locator
UTF-8	Unicode Transformation Format 8 bits
VoIP	Voice over IP
VSAT	Very Small Aperture Terminal

Liste des figures

Figure 1 : Diagramme de cas d'utilisation	24
Figure 2 : Diagramme de classe	25
Figure 3 : Diagramme de Séquence : Se connecter	30
Figure 4 : Diagramme de Séquence : Réservation	31
Figure 5 : Diagramme de Séquence : achatTicket	32
Figure 6 : Page de connexion	38
Figure 7 : Page d'inscription	38
Figure 8 : Page principale	39
Figure 9 : Page principale : option réservation	40
Figure 10 : Page de notifications	40

Introduction générale

Dans un contexte où la transformation numérique devient un enjeu majeur pour les institutions, l'optimisation des services internes constitue un levier essentiel pour améliorer le fonctionnement des structures éducatives. C'est dans cette logique que s'inscrit le projet porté par l'École des Mines, de l'Industrie et de la Géologie (EMIG), visant à moderniser la gestion de son service de restauration.

Jusqu'à présent, ce service reposait sur des procédures essentiellement manuelles. Si ces méthodes répondaient partiellement aux besoins, elles présentaient plusieurs limites : pertes de temps liées aux files d'attente, erreurs de réservation, difficultés dans la gestion des stocks et absence d'outils statistiques fiables pour appuyer les prises de décisions de l'administration.

Pour répondre à ces enjeux, **un projet de développement d'une plateforme numérique complète a été initié, articulé autour de deux volets complémentaires**

- **Une application mobile**, conçue pour simplifier l'accès des étudiants aux services du restaurant, notamment la réservation des repas et l'achat de tickets.
- **Une interface web**, destinée à l'administration, permettant notamment la gestion des tickets, des réservations et des stocks alimentaires ;

Le présent rapport porte principalement sur le travail réalisé dans le cadre du stage qui vient compléter l'initiative globale de modernisation, en offrant une interface intuitive et accessible aux utilisateurs finaux.

Ce travail s'inscrit donc dans une démarche globale d'innovation numérique pour l'EMIG, avec l'ambition de proposer un système unifié et performant au service de l'ensemble des acteurs concernés.

1. Présentation de l'entreprise d'accueil : Novatech

1.1 Historique et contexte de création

Créée en 2014 par M. Daouda HAMADOU, NOVATECH est une entreprise nigérienne spécialisée dans les Technologies de l'Information et de la Communication (TIC). Sa création s'inscrit dans une volonté claire de contribuer activement à la transformation numérique du Niger et, plus largement, du continent africain. NOVATECH est née d'un constat simple mais fondamental : dans un monde de plus en plus connecté, le numérique devient un levier incontournable de développement économique, social et culturel.

L'entreprise évolue dans un environnement marqué par l'évolution rapide des technologies, mais aussi par des défis spécifiques liés aux réalités socio-économiques locales. C'est précisément pour répondre à ces besoins que NOVATECH a développé des solutions innovantes et adaptées aux contextes africains.

Depuis sa création, NOVATECH s'est progressivement imposée comme un acteur de référence dans le domaine des TIC au Niger, en mettant son expertise au service des institutions publiques, des entreprises privées, des ONG, des organisations internationales ainsi que du grand public.

1.2 Vision, missions et valeurs

La vision de NOVATECH est de devenir « le partenaire fiable et durable » de tous ses clients, en leur proposant des solutions technologiques innovantes, efficaces et adaptées à leurs besoins. Elle ambitionne d'accompagner durablement la transition numérique dans des secteurs essentiels comme l'éducation, l'agriculture, la santé, l'environnement ou encore les services.

Pour concrétiser cette vision, NOVATECH s'est donné plusieurs missions principales :

- Fournir des services numériques de haute qualité, accessibles et conformes aux exigences des clients.
- Participer activement à la transformation digitale du Niger et de l'Afrique.
- Contribuer à la création de richesses et d'emplois durables à travers le développement d'un secteur numérique performant.
- Encourager la participation citoyenne et l'intégration sociale par des initiatives numériques accessibles à tous.

Ces engagements sont soutenus par des valeurs fortes, qui constituent le socle de la culture d'entreprise de NOVATECH :

- Qualité : offrir des prestations à forte valeur ajoutée, centrées sur les besoins réels des clients.
- Respect des délais : respecter les engagements pris afin d'optimiser les résultats et la productivité.

- Optimisation des ressources : proposer des solutions efficaces, adaptées aux contraintes budgétaires des clients.
- Capital humain : investir continuellement dans la formation des collaborateurs.
- Innovation : privilégier des solutions modernes et performantes.
- Intégrité et transparence dans toutes les démarches professionnelles.
- Réactivité et disponibilité pour un accompagnement efficace et personnalisé.

1.3 Organisation et fonctionnement

Afin d'assurer un fonctionnement efficace et une gestion optimale des projets, NOVATECH est organisée autour de quatre services principaux placés sous la responsabilité de la direction générale :

1. Service Systèmes, Réseaux et Sécurité : chargé de concevoir, installer et maintenir des infrastructures réseaux sécurisées pour l'entreprise et ses clients. Ce service est également responsable de la gestion des équipements informatiques, de la maintenance et de la veille technologique.
2. Service Développement Web et Mobile : développe des applications web et mobiles, à la fois pour les clients et pour les projets internes de NOVATECH. Ce service est à l'écoute des besoins des entreprises pour concevoir des solutions sur mesure et innovantes.
3. Service Commercial, Marketing et Administration : chargé d'assurer la relation client, la promotion des services de l'entreprise, la gestion commerciale, la fidélisation des clients et l'élaboration des offres commerciales.
4. Service Formation : propose des formations techniques dans différents domaines des TIC, destinées aux professionnels, aux entreprises et aux jeunes diplômés, afin de renforcer les compétences locales dans le secteur numérique.

1.4 Domaines d'expertise

NOVATECH propose une large gamme de services dans le secteur numérique :

- Développement d'applications web et mobiles
- Conception et hébergement de sites web
- Mise en place d'infrastructures réseaux (câblés et sans fil)
- Installation et maintenance de serveurs physiques et cloud
- Sauvegarde, sécurité informatique et continuité des services
- Infogérance des systèmes d'information
- Solutions de télécommunications (VSAT, ToIP, VoIP)

- Solutions électroniques (vidéosurveillance, contrôle d'accès, domotique)
- Solutions énergétiques pour salles serveurs et data centers
- Formations professionnelles en informatique, réseaux et télécoms

En plus de ces services classiques, NOVATECH développe également des compétences dans des domaines innovants comme :

- Data Science : gestion, traitement et valorisation des données.
- Intelligence Artificielle (IA) et Big Data : mise en place de solutions intelligentes pour accompagner la prise de décision.
- Énergie et électronique : solutions d'équipements pour la sécurisation des installations techniques.

1.5 Conclusion du Chapitre 1

Aujourd'hui, NOVATECH s'affirme comme un acteur clé de la transition numérique au Niger. Grâce à une stratégie orientée vers la qualité, l'innovation et l'adaptation aux besoins locaux, l'entreprise poursuit sa croissance en consolidant sa position de partenaire technologique fiable. Son ambition est claire : continuer à accompagner ses clients vers un avenir numérique plus performant, plus sécurisé et plus durable.

2. Présentation générale de l'EMIG

2.1 Historique et statut de l'EMIG

L'École des Mines, de l'Industrie et de la Géologie (EMIG) a été créée le 19 décembre 1982 à Yamoussoukro (Côte d'Ivoire), conformément à la Résolution n°22/78/CE du 27 octobre 1978, prise lors de la 4^e Conférence des Chefs d'État de la Communauté Économique de l'Afrique de l'Ouest (CEAO) à Bamako.

Dotée de la personnalité morale, ainsi que de l'autonomie financière et administrative, l'EMIG a été rétrocédée à l'Etat du Niger depuis le 31 Juillet 1995, suite à la dissolution de la CEAO en mars 1994.

Depuis sa réouverture le 04 février 1997, l'EMIG est un Établissement Public d'Enseignement Supérieur et Technologique, initialement sous la tutelle du Ministère des Mines, avant de passer au Ministère en charge de l'Enseignement Supérieur.

Par la loi n°2004-19 du 16 mai 2004, l'Assemblée Nationale a érigé l'EMIG en un Établissement Public à Caractère Scientifique, Culturel et Technique (EPSCT), lui conférant la personnalité morale et l'autonomie académique, scientifique, administrative et financière.

2.2 Fonctionnement et organisation de l'EMIG

L'EMIG est placée sous la tutelle du Ministère chargée de l'Enseignement Supérieur de la République du Niger.

La gouvernance de l'EMIG est assurée par :

- **Un organe délibérant :**
 - Le **Conseil d'Administration**.
- **Des organes consultatifs :**
 - Le **Conseil Scientifique** ;
 - Le **Conseil Pédagogique**.
- **Un organe exécutif :**
 - La **Direction Générale**.

L'organisation de l'organe exécutif repose sur la Direction Générale qui coiffe : (en Juin 2025)

- La **Direction des Études** (aspects académiques) ;
- La **Direction des Moyens Généraux et de la Cité** (gestion des infrastructures et de la vie en cité) ;
- La **Direction des Affaires Administratifs et financiers** (poste vacant à la date du 20-06-2025).

Elle est soutenue par plusieurs structures :

- Le Département Enseignement Généraux ;
- Le Département Génie Electrique ;
- Le Département Génie Informatique ;
- Le Département Génie Mécanique ;
- Le Département Génie Minier et Environnement ;
- Le Département Géoscience ;
- Le Service Coopération et Développement ;
- Le Service Financier et Comptable ;
- Le Service de Formation Continue et Stage ;
- Le Service de Gestion des Ressources Informatiques ;
- Le Service Intérieur de la Cité ;
- Le Service Maintenance ;
- Le Service du Personnel ;
- Le Service Restaurant ;
- Le Service de la Scolarité.

2.3 La formation initiale

L'EMIG est une institution d'enseignement supérieur nigérienne ouverte aux pays voisins, qui forme :

- Des **Ingénieurs** avec le grade Master ;
- Des **Techniciens Supérieurs** avec le grade Licence.

Les diplômés sont formés à concevoir et initier des projets, s'impliquant ainsi dans la transformation de leur environnement à l'aide de solutions adaptées aux contextes locaux.

L'Etablissement a adopté depuis 2019, pour sa formation, le système LMD.

Dans le système LMD (Licence-Master-Doctorat), une année universitaire est découpée en deux semestres. Chaque semestre est composé d'Unités d'Enseignements (UE) qui contiennent des Elément Constitutifs d'Unités d'Enseignement (ECUE) tous régis et validés par le Règlement Intérieur des Etudes de l'EMIG.

Les domaines de formation à l'EMIG sont scindés en deux grandes catégories :

- Les sciences et technologies de l'industrie comportant les filières :
 - Génie Électrique ;

- Génie Mécanique ;
- Génie Informatique.
- Les sciences de la terre regroupant les filières :
 - Génie Civil ;
 - Géologie ;
 - Génie Minier et Environnement ;
 - Science de l'Eau.

Toutes ces filières sont constituées de plusieurs options comme définies dans le Règlement Intérieur des Etudes.

2.4 Présentation du Département Génie Informatique

Le Département Génie Informatique a pour mission de former des ingénieurs avec grade master et des techniciens supérieurs avec grade licence en Informatique.

En termes de personnel, le Département compte :

- Une (1) Cheffe de Département ;
- Un (1) Chef de Département adjoint (poste vacant en juin 2025) ;
- Quatre (4) enseignants permanents, la Cheffe de Département comprise;
- Une dizaine d'enseignants vacataires.

Le Département dispose de 4 salles :

- 2 salles destinées aux élèves en cycle master (M1 et M2) ;
- 2 salles destinées aux élèves en cycle licence (L2 et L3).

2.5 Description de la Direction des Moyens Généraux et de la Cité et du Service Restaurant

2.5.1 Description de la Direction des Moyens Généraux et de la Cité

La Direction des Moyens Généraux et de la Cité est chargée de concevoir, d'élaborer et de superviser les stratégies et programmes de mise à disposition des moyens et services nécessaires au fonctionnement de l'école ; de coordonner les activités du Service Restaurant, du Service Intérieur de la Cité et du Service Maintenance.

2.6.2 Description du Service Restaurant

Le Service Restaurant est sous la responsabilité du **Chef de Service Restaurant**, qui relève du **Directeur des Moyens Généraux de la Cité**. Ce service garantit des repas en qualité et en quantité aux élèves.

Le service de restaurant de l'EMIG joue un rôle fondamental dans la vie quotidienne des étudiants, fournissant un service essentiel pour leur bien-être et leur concentration académique. Ce service est un point central de la vie étudiante, et son efficacité impacte directement l'expérience globale des usagers de l'institution.

Les membres clés incluent :

- **Un (1) Vendeur de Tickets** : Gère des ventes de tickets pour l'accès aux repas;
- **Un (1) Magasinier** : Gère les stocks des produits de consommation et des provisions ;
- **Un (1) Responsable de guichet** : Coordonne des réservations ;
- **Cuisiniers** : Se chargent de la préparation des repas ;
- **Autres membres** : Chargés de la préparation, du service, du nettoyage, de la manutention etc.

Le service restaurant dispose de :

- **Un (1) Restaurant interne** : avec une capacité de **500 couverts**,
- **Une (1) Cuisine équipée de** :
 - Fourneaux et marmites ;
 - Ustensiles divers.
- **Moyen de stockage** :
 - Deux réfrigérateurs ;
 - Magasin de provisions et une balance pour la gestion précise des quantités ;
 - Une chambre froide.
- **Trois (3) Bureaux dédiés** :
 - Un (1) pour le Vendeur des tickets ;
 - Un (1) pour le Responsable de guichet et le Magasinier ;
 - Un pour le Chef Service Restaurant.
- **Une Cafétéria** : qui est gérée par un prestataire extérieur sous contrat.

3. Concepts de Base

3.1 Introduction

Avant de présenter la conception de la plateforme, il est utile de revenir brièvement sur certains concepts fondamentaux liés aux applications web et mobiles. Ces notions permettent de mieux comprendre la logique générale du système développé et de situer les choix techniques qui ont été faits.

La compréhension de ces concepts constitue ainsi une base nécessaire pour aborder la suite du projet, en particulier en ce qui concerne l'organisation, le fonctionnement et l'interaction entre les différentes parties de l'application.

3.2 Définition des applications web

Elles sont accessibles via un navigateur web et fonctionnent sur Internet. Elles sont généralement développées à l'aide de technologies web (HTML, CSS, JavaScript) et peuvent être utilisées sur différents appareils (ordinateurs, smartphones, tablettes) sans nécessiter d'installation préalable.

3.3 Définition des applications mobiles

Une application mobile est un logiciel spécifiquement conçu pour être utilisé sur des appareils tels que les smartphones et les tablettes. Ces applications sont optimisées pour tirer parti des fonctionnalités uniques de ces appareils, comme l'accès rapide à l'information, une navigation intuitive, et l'interaction avec les capteurs intégrés (GPS, caméra, etc.).

Il existe plusieurs types d'applications mobiles, chacune avec ses propres caractéristiques et avantages :

- **Applications natives** : Développées spécifiquement pour un système d'exploitation donné (Android ou iOS) en utilisant des langages et des outils propres à chaque plateforme (par exemple, Kotlin/Java pour Android, Swift/Objective-C pour iOS). Elles offrent les meilleures performances, une fluidité optimale et un accès complet à toutes les fonctionnalités du téléphone.
- **Applications hybrides** : Conçues avec des technologies web (HTML, CSS, JavaScript) et ensuite encapsulées dans un conteneur natif. Elles permettent de fonctionner sur plusieurs plateformes à partir d'un seul code source, mais peuvent être légèrement moins performantes que les applications natives. Des exemples de frameworks incluent Ionic et Cordova.
- **Applications web mobiles** : Il s'agit de sites web optimisés pour les mobiles, accessibles via un navigateur. Elles ne nécessitent pas d'installation mais offrent une expérience utilisateur moins fluide et dépendent de la connexion internet.
- **Applications multiplateformes** : Développées à partir d'un unique code source pour plusieurs systèmes d'exploitation grâce à des frameworks comme Flutter ou React Native. Elles cherchent à combiner la bonne performance des

applications natives avec la réduction des coûts de développement des applications hybrides.

- **Progressive Web Apps (PWA)** : Ce sont des applications web évoluées qui offrent une expérience utilisateur proche de celle des applications mobiles traditionnelles. Elles peuvent fonctionner hors ligne, envoyer des notifications push et être installées sur l'écran d'accueil de l'utilisateur, tout en restant accessibles depuis un navigateur. Elles représentent un compromis intéressant entre les avantages du web et du mobile.

3.4 Les systèmes d'exploitation mobiles (Android, iOS)

Les deux systèmes d'exploitation dominants sur le marché des appareils mobiles sont : Android, développé par Google, et iOS, développé par Apple. Le choix de l'un ou l'autre a des implications directes sur le processus de développement d'applications mobiles.

Android est un système d'exploitation ouvert et flexible, ce qui a favorisé son adoption par une grande variété de fabricants d'appareils. Cette ouverture lui confère une part de marché très importante à l'échelle mondiale. Son écosystème est vaste et diversifié, offrant aux développeurs une grande liberté en termes d'outils et de personnalisation.

iOS, en revanche, est un système propriétaire exclusivement conçu pour les appareils Apple (iPhone, iPad). Il est réputé pour sa fluidité, sa sécurité robuste et son écosystème fermé et intégré. Cette approche garantit une expérience utilisateur cohérente et optimisée sur l'ensemble des appareils Apple.

Le choix entre Android et iOS pour le développement d'une application impacte directement les environnements de développement, les langages de programmation (par exemple, Kotlin/Java pour Android, Swift/Objective-C pour iOS) et les outils spécifiques à utiliser. Cependant, les frameworks multiplateformes comme Flutter ou React Native permettent de cibler les deux systèmes d'exploitation à partir d'un code source unique, offrant un compromis efficace pour atteindre une large audience tout en optimisant les coûts de développement.

3.5 Avantages des applications mobiles dans un contexte universitaire

L'intégration d'applications mobiles dans un environnement universitaire comme l'EMIG a été reconnue comme offrant des avantages stratégiques qui vont au-delà de la simple commodité :

- **Accessibilité accrue** : Les étudiants peuvent accéder aux services de réservation du restaurant à tout moment, directement depuis leur appareil personnel. Cela élimine les contraintes de temps et de localisation associées aux systèmes manuels.
- **Gain de temps significatif** : La digitalisation des processus permet de réduire drastiquement le temps nécessaire pour des tâches telles que la réservation de repas, la consultation des menus ou la gestion des comptes. Les files

d'attente sont réduites, libérant du temps précieux pour les activités académiques et personnelles des étudiants.

- **Communication améliorée et en temps réel** : Les applications mobiles facilitent l'envoi de notifications push et de mises à jour en temps réel. Cela renforce l'interaction entre le service de restauration et les étudiants, permettant d'informer rapidement sur les changements de menu, les annulations de service ou les rappels de réservation.
- **Modernisation de l'image de l'EMIG** : L'adoption de technologies mobiles avancées positionne l'EMIG comme une institution moderne et innovante. Cela valorise son image auprès des étudiants actuels et potentiels, démontrant son engagement envers l'amélioration continue de ses services et l'intégration des outils numériques du 21^e siècle.

Ces avantages combinés contribuent à une expérience étudiante plus fluide, plus efficace et plus satisfaisante.

3.6 Comparaison entre les applications mobiles et web

Une comparaison des deux approches, application mobile et application web est essentielle, car le choix dépend des objectifs spécifiques du projet et de l'expérience utilisateur souhaitée. Voici une comparaison des critères clés :

Critère	Application mobile	Application web
Accessibilité	Téléchargée et installée depuis un store (App Store, Google Play).	Accessible via un navigateur web (Chrome, Firefox, Safari).
Performance	Généralement plus rapide et plus fluide, avec un accès complet aux fonctionnalités matérielles du téléphone (GPS, caméra, notifications push).	Moins rapide, dépend fortement de la qualité de la connexion internet et des capacités du navigateur. Accès limité aux fonctionnalités du téléphone.
Développement	Peut être spécifique à chaque OS (natif) ou multiplateforme (avec des frameworks comme Flutter, React Native) à partir d'un code unique.	Développement unique pour tous les appareils et systèmes d'exploitation, car il s'agit d'un site web optimisé.
Mise à jour	Nécessite le téléchargement et l'installation des mises à jour par l'utilisateur via le store.	Automatique, sans intervention de l'utilisateur, car le contenu est mis à jour sur le serveur.
Coût	Généralement plus élevé, surtout pour le développement natif sur plusieurs plateformes.	Moins élevé en raison du développement unique et de l'absence de processus de publication sur les stores.

3.7 Conclusion du Chapitre 3

L'exploration des concepts fondamentaux liés aux applications web et mobiles permet de mieux appréhender les choix techniques opérés dans le cadre du développement de la plateforme de gestion du service restauration de l'EMIG. Cette mise en contexte est essentielle pour comprendre la structure générale du projet, ses enjeux techniques, ainsi que les raisons ayant conduit à privilégier certaines technologies plutôt que d'autres.

4. Analyse et Spécification des Besoins

4.1 Introduction

Le service de restaurant de l'EMIG, bien qu'essentiel à la vie étudiante, est confronté aux limites inhérentes à son système manuel actuel, qui impactent sa fiabilité, son efficacité et son accessibilité. La nécessité de moderniser ce service par le biais d'une plateforme numérique web et mobile est devenue impérative. Cette transformation vise à automatiser les réservations, à digitaliser la gestion des tickets et à améliorer la communication entre toutes les parties prenantes.

4.2 Étude de l'existant

L'analyse de l'organisation actuelle du service restaurant de l'EMIG a constitué le point de départ des travaux. Cela a permis de mettre en évidence les acteurs impliqués, les processus en place et les dysfonctionnements majeurs.

4.2.1 Organisation actuelle du service restaurant

L'organisation actuelle du service restaurant de l'EMIG est caractérisée par des procédures manuelles et implique plusieurs acteurs clés identifiés :

- **Chef du Service Restaurant** : Supervise l'organisation globale du service.
- **Vendeur de Tickets** : Vend les tickets de repas et enregistre les transactions.
- **Magasinier** : Gère les stocks alimentaires, effectue les inventaires et réceptionne les commandes.
- **Responsable de Guichet** : Gère les fiches de réservation des repas étudiants.
- **Étudiant** : Achète des tickets et Effectue les réservations de repas
- **Directeur des Moyens Généraux de la Cité (DMGC)** : Valide les marchés à commande et assure le suivi administratif.

Les processus clés actuels sont les suivants :

- **Réservations** : Quotidiennes pour le petit-déjeuner, le déjeuner et le dîner.
- **Gestion des stocks** : Basée sur des fiches d'entrée et de sortie papiers des produits de consommation, avec des inventaires mensuels.
- **Distribution de tickets** : La gestion des stocks de tickets est manuelle, et le vendeur remet régulièrement les recettes au niveau de la comptabilité de l'EMIG.
- **Marchés à commande** : Contrats annuels divisés en six lots, renouvelables une fois.

Le flux simplifié des opérations est le suivant :

1. **L'étudiant** achète son ticket auprès du **Vendeur de ticket**.
2. Il présente son ticket et indique les jours pour lesquels il souhaite faire une réservation, ces dernières seront reportées sur une fiche papier par le **Responsable de Guichet**.
3. **Le Responsable de guichet** comptabilise le nombre de réservations faites et transmet le total au Magasinier pour un jour spécifique.
4. **Le Magasinier** effectue une estimation des quantités alimentaires à préparer en fonction des réservations enregistrées et les transmet au **Cuisinier**.
5. La cuisine prépare les repas selon les quantités transmises.

4.3 Critique de l'existant

4.3.1 Limites et contraintes observées

L'analyse du fonctionnement actuel a révélé des problèmes majeurs et des contraintes ayant un impact négatif sur la qualité de service et l'efficacité opérationnelle. Le tableau ci-dessous résume ces problématiques et leurs conséquences :

Problèmes / Contraintes	Conséquences négatives
Processus 100 % manuel	Erreurs fréquentes dans les réservations, lenteur dans le traitement, surcharge de travail pour le personnel.
Fiches papier fragiles	Risque élevé de perte, dégradation ou altération des données ; erreurs de saisie fréquentes.
Absence de suivi en temps réel	Impossible d'adapter rapidement la préparation des repas aux fluctuations des réservations.
Communication éclatée entre acteurs	Retards dans les transmissions d'informations, incohérences entre les données collectées, mauvaise coordination globale.
Gestion manuelle des stocks	Erreurs d'inventaire, gaspillage alimentaire, retards dans le renouvellement des produits.
Étudiant pouvant se tromper de numéro lors de la réservation	Risque d'erreurs dans la prise en compte des repas réservés.
Perte de temps pour l'étudiant	L'étudiant peut rater une partie de ses cours en raison du temps perdu pour réserver ou acheter un ticket.
Files d'attente longues	Files d'attente à la fois pour le paiement des tickets et pour la réservation sur fiches papier.
Absence du vendeur de tickets ou du responsable de guichet à certaines heures	Interruption du service de réservation et difficultés pour les étudiants à finaliser leur demande.
Indisponibilité de tickets	Les étudiants peuvent trouver le stock de tickets épuisé temporairement, les empêchant de réserver leurs repas.

Manque de reçu lors de l'achat de ticket	Aucune preuve d'achat formelle pour les étudiants, compliquant les éventuelles vérifications.
Problèmes de monnaie	Retards et insatisfaction lors des paiements en raison d'un manque de monnaie disponible au guichet.

Ce fonctionnement intensif et manuel, reposant sur des supports physiques fragiles, ralentit considérablement l'efficacité du service, générant des erreurs critiques et pénalisant directement le personnel du restaurant ainsi que les étudiants.

4.4 Solution proposée

La solution envisagée pour remédier aux lacunes du système actuel est la création d'une plateforme Web et Mobile pour la gestion du service restaurant de l'EMIG. Cette plateforme est conçue pour apporter une modernisation significative et des avantages opérationnels considérables.

Les objectifs principaux de cette solution sont les suivants :

- **Accessibilité Mobile** : Fournir une application mobile dédiée aux étudiants, leur permettant de réserver facilement leurs repas.
- **Centralisation** : Établir une base de données unique et centralisée pour toutes les informations relatives aux réservations, tickets, stocks, historiques et rapports etc. Cela éliminera les silos d'information et les incohérences.
- **Automatisation** : Automatiser les calculs de quantités de produits alimentaires nécessaires, l'enregistrement des réservations et la génération d'alertes pour les seuils critiques de stock ou les produits périmés.
- **Sécurité et Fiabilité** : Assurer une authentification sécurisée, des sauvegardes régulières des données et le chiffrement des informations sensibles pour garantir l'intégrité et la confidentialité.

Les fonctionnalités principales de la plateforme se déclinent en plusieurs modules :

- **Module de gestion des réservations (application Web)** :
 - Calcul automatique du nombre de réservations
- **Module de gestion des tickets (application Web)** :
 - Vente de tickets en ligne avec génération de QR codes.
 - Suivi précis des transactions et réconciliation des ventes ;
 - Module de création de ticket dématérialisé.
- **Module de gestion des stocks (application Web)** :
 - Enregistrement des entrées et sorties de produits en temps réel.

- Détection automatique des seuils critiques de stock.
- Génération d'alertes pour les produits avariés ou les ruptures de stock imminentes.
- **Portail étudiant (application mobile) :**
 - Réservation rapide des repas en quelques clics.
 - Consultation de l'historique des réservations.
 - Possibilité d'annuler ou une réservation avant une date limite définie.
 - Réception de notifications (push, SMS ou email) pour les confirmations ou les rappels.
 - Paiement des tickets, y compris l'intégration de QR codes de validation pour faciliter le processus.
- **Rapports et Statistiques :**
 - Génération et exportation de rapports au format PDF ou Excel concernant les réservations, les ventes, les inventaires et les alertes générées.
 - Tableau de bord d'administration avec des statistiques visuelles pour une meilleure prise de décision.

4.5 Acteurs du système et leurs rôles

La mise en place de la nouvelle plateforme redéfinit les rôles des différents acteurs impliqués dans le service restaurant de l'EMIG, en optimisant leurs interactions avec le système.

Acteur	Rôle dans la plateforme
Administrateur	Gestion globale des utilisateurs, génération de rapports.
Chef du Service Restaurant	Supervision des activités du restaurant, validation des rapports.
Magasinier	Suivi et mise à jour du stock des produits en temps réel, gestion des produits alimentaires, génération d'alertes de seuil critique.
Vendeur de Tickets	Gestion des ventes de tickets (y compris en ligne), génération de QR codes, réconciliation des ventes.
Responsable de Guichet	Vérification et suivi des réservations étudiants, gestion des accès aux repas.
Étudiant	Réservation et annulation de repas via l'application mobile, consultation de l'historique.

4.6 Besoins fonctionnels

Les besoins fonctionnels de la plateforme de gestion des services du restaurant de l'EMIG sont définis pour répondre aux exigences opérationnelles et améliorer l'expérience utilisateur :

- **Gestion multi-acteurs avec rôles et permissions** : Le système doit permettre une gestion différenciée des accès et des fonctionnalités en fonction du rôle de chaque utilisateur (administrateur, étudiant, magasinier, vendeur de tickets, responsable de guichet, chef de service).
- **Réservation de repas via web ou mobile** : Les étudiants doivent pouvoir réserver leurs repas de manière intuitive, que ce soit via l'application mobile.
- **Notifications automatiques** : Le système doit envoyer des notifications (push mobile, email, SMS) pour confirmer les réservations.
- **Historique et suivi personnalisé pour chaque étudiant** : Chaque étudiant doit pouvoir consulter son historique de réservations, ainsi que son solde de tickets restants.
- **Génération de rapports dynamiques** : La plateforme doit être capable de générer des rapports personnalisables (ventes, réservations, inventaires) pour l'analyse et la prise de décision.

4.7 Besoins non fonctionnels

Outre les fonctionnalités, il est crucial que la solution réponde à des exigences non fonctionnelles essentielles pour sa robustesse et son adoption :

- **Sécurité** : Une authentification forte est requise, utilisant des tokens JWT et potentiellement l'authentification à deux facteurs (OTP SMS). Toutes les communications devront être cryptées via HTTPS pour protéger les données sensibles.
- **Performance** : Les interfaces doivent se charger rapidement et les temps de réponse des requêtes API doivent être optimisés pour garantir une expérience utilisateur fluide, même en période de forte affluence.
- **Compatibilité** : La plateforme web doit être responsive et compatible avec les navigateurs récents (Chrome, Edge, Firefox). L'application mobile doit être compatible avec les principaux systèmes d'exploitation (Android, iOS).
- **Scalabilité** : La solution doit être conçue pour gérer une augmentation progressive du nombre d'utilisateurs et de transactions sans dégradation significative des performances.

4.8 Contraintes techniques

Les choix techniques pour le développement de la plateforme ont été guidés par des contraintes spécifiques :

- **Accès Internet** : Un accès Internet stable est nécessaire pour tous les utilisateurs (étudiants et personnel administratif) afin d'interagir avec la plateforme.
- **Plateformes ciblées** :
 - **Application Web** : Le frontend sera développé en ReactJS avec TailwindCSS pour le style et Axios pour les requêtes HTTP. Le backend sera construit avec Django REST Framework.
 - **Application Mobile** : Développée avec React Native, pour sa capacité à générer des applications natives pour Android et iOS .
- **Hébergement** : La plateforme nécessitera un serveur cloud sécurisé pour l'hébergement (par exemple, AWS ou DigitalOcean), garantissant disponibilité et évolutivité.
- **Base de données** : Un système de gestion de base de données relationnel robuste est requis, avec PostgreSQL comme choix privilégié pour sa fiabilité et ses performances.
- **Authentification** : L'authentification des utilisateurs sera gérée par JWT (JSON Web Tokens) via SimpleJWT pour une sécurité standardisée.
- **Génération de PDF** : Des bibliothèques comme ReportLab seront utilisées pour la génération des reçus et rapports au format PDF.
- **API REST** : Une API RESTful sera le canal de communication principal entre l'application mobile, l'application web et le backend, assurant une interopérabilité efficace.

4.9 Présentation du projet global

Le projet consiste à concevoir et développer une **plateforme complète de gestion du service restaurant de l'EMIG**. Cette plateforme a pour objectif de moderniser le fonctionnement du service en remplaçant les méthodes manuelles par un système numérique plus efficace, fiable et accessible.

La plateforme est structurée autour de **deux volets principaux** :

- **Une application mobile** dédiée aux étudiants, leur offrant :
 - L'achat dématérialisé de tickets ;
 - La réservation rapide des repas via une interface simple et intuitive ;
 - La réception de notifications utiles (confirmation de réservation, rappels, informations diverses).
- **Une interface web** destinée à l'administration du restaurant. Cette interface permet notamment de :
 - Gérer la vente des tickets ;

- Enregistrer et suivre les réservations ;
- Gérer les stocks alimentaires ;
- Produire des statistiques utiles à la prise de décision ;

4.9.1. Contexte du développement

La conception de cette plateforme a été amorcée dans le cadre **d'un premier projet réalisé en amont du stage**, avec une **première version de l'application web** dédiée principalement à la gestion par l'administration. Ce développement constitue le socle sur lequel vient se greffer le volet mobile.

4.9.2. Périmètre du travail réalisé durant le stage

Durant le stage, l'essentiel du travail s'est concentré sur le développement de l'application mobile. Ce travail a consisté à :

- Concevoir et développer l'interface mobile à destination des étudiants ;
- Intégrer les fonctionnalités de réservation de repas et d'achat de tickets ;
- Mettre en place la communication entre l'application mobile et le backend existant ;
- Réaliser certaines adaptations et améliorations sur l'application web, afin d'assurer une cohérence fonctionnelle entre les deux interfaces.

L'objectif principal a été de fournir une solution numérique cohérente et adaptée à l'ensemble des acteurs impliqués dans la gestion du service restauration :

- **Les étudiants**, utilisateurs finaux de l'application mobile ;
- **Le personnel administratif**, utilisateurs finaux de l'interface web.

Ce travail a donc permis d'avancer dans la démarche globale de modernisation du service restauration de l'EMIG, en assurant la complémentarité entre les deux volets de la plateforme.

4.10 Objectifs généraux et spécifiques

Le projet de stage, axé sur la **conception et la réalisation d'une plateforme numérique complète** pour la gestion du service restaurant de l'EMIG, a pour **objectif global de développer une solution web et mobile intégrée**. Cette plateforme vise à transformer en profondeur les opérations du service restauration en remplaçant les procédures manuelles par un système numérique centralisé, fiable et accessible.

4.10.1 Objectifs spécifiques

Pour atteindre cet objectif global, plusieurs objectifs spécifiques ont été définis :

- **Optimiser la gestion pour les responsables de la restauration** : la plateforme doit permettre une gestion centralisée et efficace des différents processus (vente de tickets, réservations, gestion des stocks) et des utilisateurs (étudiants, agents du services, administrateurs).

- **Fluidifier la réservation des plats par les étudiants** : une application mobile dédiée doit offrir aux étudiants la possibilité de réserver leurs repas rapidement et facilement via leur smartphone. L'application doit également permettre la consultation de l'historique des réservations et la possibilité d'annuler une réservation avant une date limite fixée.
- **Automatiser les rapports et les reçus** : la plateforme doit être capable de générer automatiquement des rapports et des reçus, sous différents formats (PDF, Excel), concernant les réservations, les ventes et les inventaires. Cette automatisation a pour but de fournir des statistiques fiables pour aider à la prise de décision.

Ces objectifs contribuent à produire des avantages fondamentaux pour le service restauration de l'EMIG :

- **Centralisation des données** : en établissant une base de données unique regroupant toutes les informations relatives aux réservations, aux tickets, aux historiques, aux rapports et aux inventaires, la plateforme élimine les risques d'incohérence et de perte d'informations liés aux méthodes manuelles.
- **Automatisation des processus** : l'enregistrement des réservations, la génération d'alertes et la création de rapports sont automatisés, ce qui réduit considérablement la charge de travail du personnel et limite les risques d'erreurs humaines.
- **Accessibilité mobile** : grâce à l'application mobile, les services sont accessibles aux étudiants à tout moment, améliorant leur autonomie et leur satisfaction.

Sécurité et fiabilité : la mise en œuvre de mécanismes d'authentification sécurisée, de sauvegardes régulières et de protection des données garantit la sécurité et la fiabilité de l'ensemble du système

4.10.2 Démarche méthodologique adoptée

Pour mener à bien ce projet, une **démarche méthodologique structurée** a été adoptée, permettant de progresser de manière systématique de l'analyse à la réalisation de la solution.

La première étape a consisté en **une analyse approfondie de l'existant**. Nous avons étudié le fonctionnement du service restaurant, en identifiant les différents acteurs impliqués (étudiants, responsables de caisse, gestionnaires de stocks), ainsi que leurs processus respectifs : réservation des repas, distribution des tickets, gestion des stocks, etc. Cette analyse a mis en évidence plusieurs dysfonctionnements majeurs, tels que :

- les erreurs de comptage des repas ;
- les pertes de temps générées par les files d'attente et l'enregistrement manuel.

Ces constats ont permis de **justifier clairement la nécessité d'une solution numérique** et d'orienter la définition des besoins fonctionnels et techniques.

À l'issue de cette analyse, une **solution numérique globale a été proposée**, incluant :

- Un portail web dédié à la gestion administrative (tickets, réservations, rapports, stocks) ;
- Une application mobile destinée aux étudiants pour simplifier les démarches de réservation.

Les fonctionnalités clés ont été définies (authentification, réservation, annulation, historique, génération de rapports) ainsi que les rôles attribués aux différents utilisateurs.

La phase suivante a été consacrée à **la conception technique de la solution**, avec une attention particulière portée à la modélisation. L'utilisation du langage UML (Unified Modeling Language) a permis de représenter clairement :

- Les cas d'utilisation pour décrire les interactions entre utilisateurs et système ;
- Les diagrammes de classes pour modéliser les différentes entités ;
- Les diagrammes de séquence pour illustrer les processus importants (authentification, réservation, achat de tickets).

Cette modélisation rigoureuse a constitué **une base solide avant le développement**, facilitant la communication entre les intervenants et garantissant la cohérence de l'architecture.

Le travail de développement a été organisé **selon une feuille de route découpée en étapes** :

Étape 1 : Stabilisation et préparation du backend (Avril)

Cette phase a été consacrée à la préparation de l'interface de communication entre l'application mobile et le backend existant. Elle comprenait :

- La correction de certaines erreurs liées à la configuration de la base de données (migrations Django) ;
- La sécurisation de l'authentification à l'aide de tokens JWT ;
- La mise en place des API REST nécessaires (réservations, historique, authentification).

Étape 2 : Développement de l'application mobile (Mai)

Le développement de l'application mobile a été effectué avec **React Native**, un choix pertinent permettant un déploiement multiplateforme (Android/iOS) avec un code unique. Cette phase a couvert :

- L'implémentation du système d'authentification par email ;
- Le développement des interfaces utilisateurs (consultation du solde de ticket, réservation de repas, historique des réservations) ;

- L'intégration des QR codes pour faciliter la validation des réservations ;
- La communication avec le backend via des appels API sécurisés.

Étape 3 : Tests et déploiement (Juin)

Des tests fonctionnels ont été réalisés avec des outils comme Postman pour vérifier le bon fonctionnement des API, ainsi que directement via l'application mobile. Cette phase a également inclus :

- Le déploiement du backend sur un serveur distant ;
- La génération de l'application mobile (APK) ;

Étape 4 : Documentation et finalisation (Juin)

La dernière étape a consisté à produire la documentation complète du projet, incluant :

- Les objectifs et choix techniques ;
- Les diagrammes UML ;
- Les captures d'écran de l'application ;
- Le code source et les annexes techniques.

Certaines fonctionnalités complémentaires ont également été envisagées pour enrichir la plateforme, telles que l'ajout de statistiques et de graphiques dédiés aux administrateurs.

Cette démarche progressive et structurée a permis d'aborder le projet de manière rigoureuse, en intégrant progressivement l'ensemble des besoins identifiés, et en appliquant les meilleures pratiques du développement logiciel.

4.11 Conclusion du Chapitre 4

Cette phase d'analyse a permis de comprendre que le service restaurant de l'EMIG est confronté à des défis importants découlant de son organisation manuelle actuelle. La mise en place d'une plateforme Web et Mobile représente une solution stratégique, non seulement pour automatiser les processus liés au service restaurant, mais aussi pour améliorer de manière significative l'expérience des étudiants et la productivité du personnel. Grâce à une solution centralisée, sécurisée et accessible, l'EMIG posera un jalon important vers la digitalisation de ses services universitaires, en phase avec les exigences modernes d'efficacité et de qualité.

5. Conception

5.1 Introduction

La phase de conception est une étape cruciale pour la réussite de ce projet logiciel complexe. Pour assurer la clarté, la cohérence et la robustesse de l'application Web et Mobile de gestion du service restaurant de l'EMIG, la modélisation UML (Unified Modeling Language) a été adoptée. UML fournit un ensemble de diagrammes standardisés qui permettent de représenter visuellement les fonctionnalités attendues, la structure statique des objets métier et le comportement dynamique du système. Cette approche a permis de garantir une description précise de chaque acteur, composant et interaction avant l'implémentation, facilitant ainsi la communication et la maintenance future du système.

5.2 Modélisation UML

UML a été utilisé, un langage de modélisation graphique standardisé par l'OMG (Object Management Group). Ses principaux avantages résident dans son indépendance technologique, ce qui signifie que les diagrammes restent valables quel que soit le langage de programmation choisi (ici Django/ReactJS/React Native), sa complétude, couvrant les aspects structurels et comportementaux, et sa capacité à servir de documentation vivante tout au long du cycle de vie du projet.

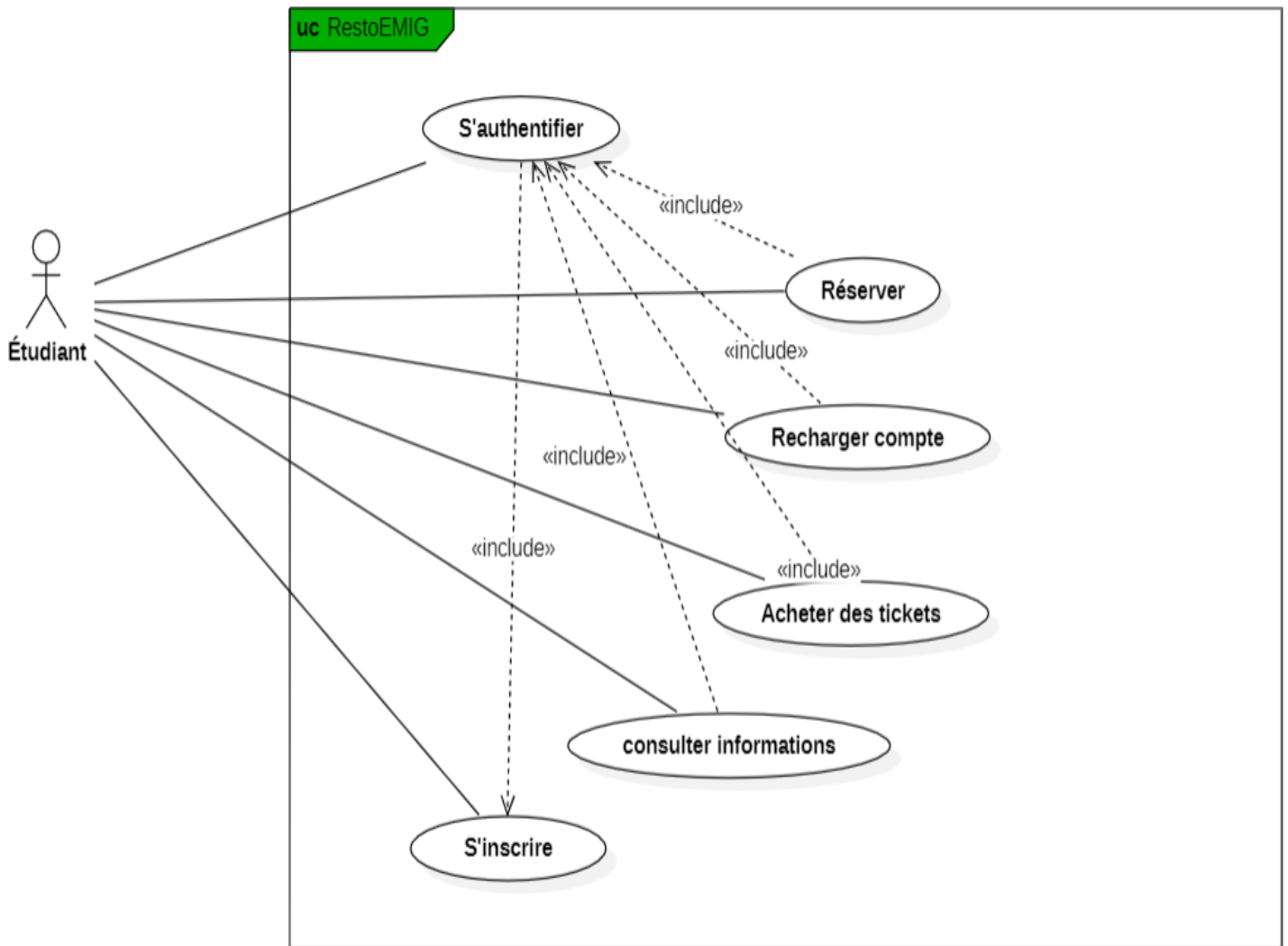


Figure 1 : Diagramme de cas d'utilisation

5.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation a pour objectif de décrire les interactions entre les utilisateurs (acteurs) et le système Web/Mobile. Il met en évidence les fonctionnalités offertes par la plateforme du point de vue de l'utilisateur.

Les points clés de ce diagramme sont les suivants :

- **L'étudiant** : Peut s'authentifier, réserver ou annuler un plat, payer des tickets et recevoir des notifications (confirmations, rappels, alertes).
- **Le personnel du restaurant** : Cela inclut le vendeur de tickets, le responsable de guichet, le chef de service, le magasinier, et l'administrateur. Ils peuvent gérer les stocks, les ventes de tickets, les réservations et générer divers rapports (ventes, stocks).

Ce diagramme a permis de visualiser clairement les périmètres fonctionnels de chaque catégorie d'utilisateur et d'assurer que tous les besoins sont couverts.

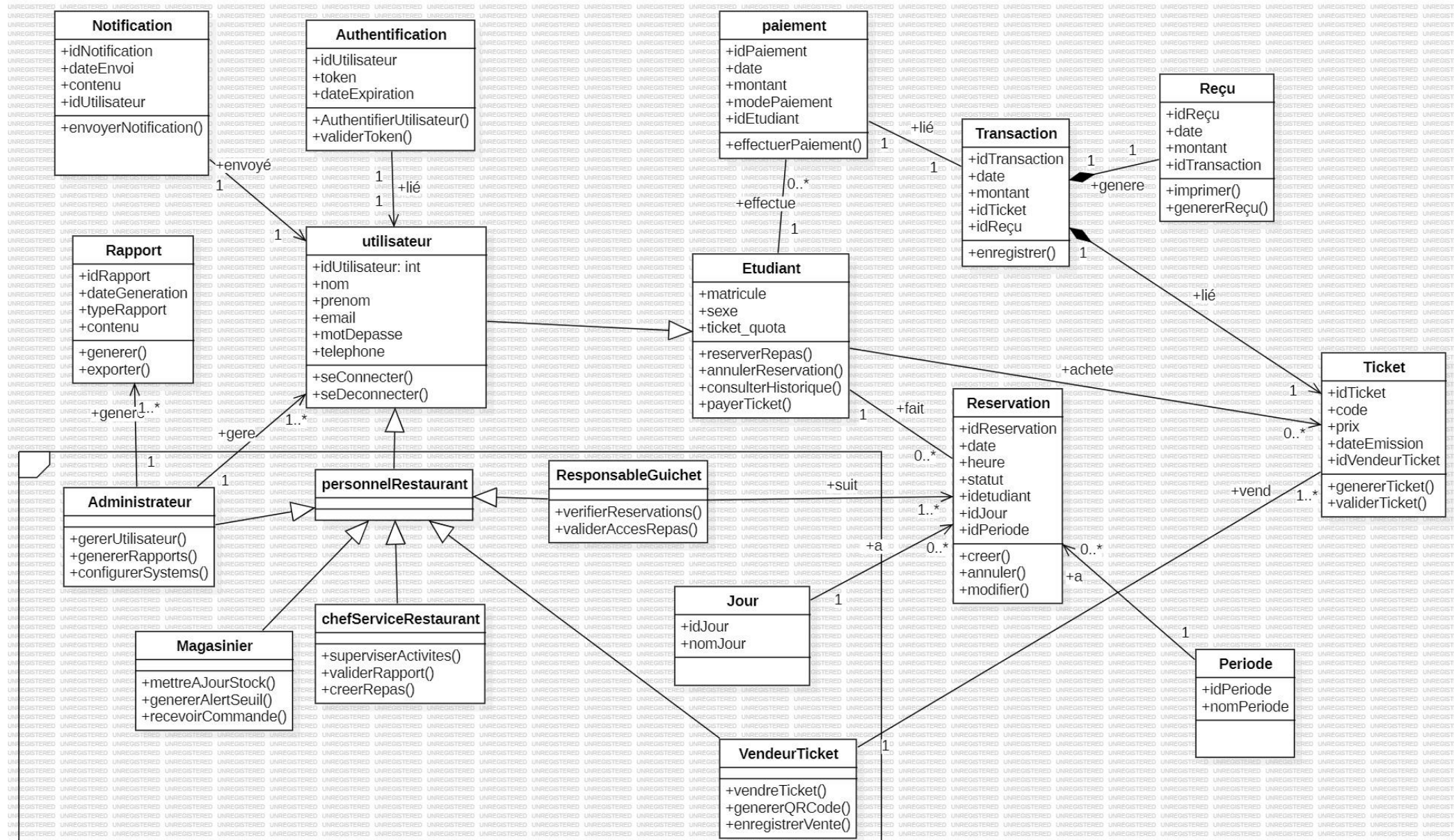


Figure 2 : Diagramme de classe

5.2.2 Diagramme de classes

Le diagramme de classes a été essentiel pour modéliser les entités métier du système, leurs attributs, leurs méthodes et les relations qui les unissent. Il offre une vue statique et structurée de la base de données et de la logique métier.

Voici les principales classes identifiées et leurs relations :

Classes et leurs attributs et méthodes :

- **Utilisateur** (Classe Abstraite)
 - Attributs : id, nom, email, motDePasse
 - Méthodes : seConnecter(), seDeconnecter()
- **Etudiant** (Hérite de Utilisateur)
 - Attributs : matricule
 - Méthodes : reserverRepas(), annulerReservation(), consulterHistorique(), payerTicket()
- **PersonnelRestaurant** (Classe Abstraite, Hérite d'Utilisateur)
 - Attributs : Aucun attribut spécifique
 - Méthodes : Aucune méthode spécifique
- **Administrateur** (Hérite de PersonnelRestaurant)
 - Attributs : Aucun attribut spécifique
 - Méthodes : gererUtilisateurs(), configurerSysteme(), genererRapports()
- **ChefServiceRestaurant** (Hérite de PersonnelRestaurant)
 - Attributs : Aucun attribut spécifique
 - Méthodes : superviserActivites(), validerRapports(), creerMenu()
- **Magasinier** (Hérite de PersonnelRestaurant)
 - Attributs : Aucun attribut spécifique
 - Méthodes : suivreStock(), mettreAJourStock(), genererAlerteSeuil()
- **VendeurTickets** (Hérite de PersonnelRestaurant)
 - Attributs : Aucun attribut spécifique
 - Méthodes : vendreTicket(), genererQRCode(), reconcilierVentres()
- **ResponsableGuichet** (Hérite de PersonnelRestaurant)
 - Attributs : Aucun attribut spécifique

- Méthodes : verifierReservations(), suivreReservations()
- **Reservation**
 - Attributs : id, date, heure, statut, etudiant_id
 - Méthodes : creerReservation(), annulerReservation(), modifierReservation()
- **Ticket**
 - Attributs : id, code, dateVente, prix, vendeur_id
 - Méthodes : genererTicket(), validerTicket()
- **Transaction** (Composition de Ticket et Recu)
 - Attributs : id, date, montant, ticket_id, recu_id
 - Méthodes : enregistrerTransaction()
- **Paieement**
 - Attributs : id, date, montant, modePaieement
 - Méthodes : effectuerPaieement()
- **Reçu** (Composition de Transaction)
 - Attributs : id, dateEmission, montant, transaction_id
 - Méthodes : genererRecu(), imprimerRecu()
- **Rapport** (Abstract)
 - Attributs : id, dateGeneration, contenu
 - Méthodes : genererRapport(), exporterRapport()
- **RapportDeVentes** (Hérite de Rapport)
 - Attributs : Aucun attribut spécifique
 - Méthodes : calculerTotalVentes()
- **Notification**
 - Attributs : id, dateEnvoi, contenu, typeNotification, utilisateur_id
 - Méthodes : envoyerNotification()
- **Authentication**
 - Attributs : utilisateur_id, token, dateExpiration
 - Méthodes : authentifierUtilisateur(), validerToken()

Relations entre les classes :

- **Héritage :**

- Utilisateur est une superclasse pour Etudiant et PersonnelRestaurant.
- PersonnelRestaurant est une superclasse pour ChefServiceRestaurant, Magasinier, VendeurTickets, ResponsableGuichet .
- Rapport est une superclasse pour RapportDeVentes .

- **Associations :**

- Administrateur gère Utilisateur (1 à plusieurs : 1 -> *).
- Administrateur génère Rapport (1 à plusieurs : 1 -> *).
- VendeurTickets vend Ticket (1 à plusieurs : 1 -> *).
- ResponsableGuichet vérifie et suit Reservation (1 à plusieurs : 1 -> *).
- Etudiant crée Reservation (1 à plusieurs : 1 -> *).
- Etudiant achète Ticket (1 à plusieurs : 1 -> *).
- Etudiant effectue Paiement (1 à plusieurs : 1 -> *).
- Reservation concerne Repas (1 à plusieurs : 1 -> *).
- Reservation est faite par Etudiant (1 à 1 : 1 -> 1).
- Ticket est lié à Transaction (1 à 1 : 1 -> 1).
- Transaction est liée à Paiement (1 à 1 : 1 -> 1).
- Transaction génère Reçu (1 à 1 : 1 -> 1).
- Paiement est lié à Transaction (1 à 1 : 1 -> 1).
- Paiement est effectué par Etudiant (1 à 1 : 1 -> 1).
- Notification est envoyée à Utilisateur (1 à 1 : 1 -> 1).
- Authentification est liée à Utilisateur (1 à 1 : 1 -> 1).

- **Composition :**

- Transaction compose Ticket et Reçu (1 à 1 : 1 -> 1 pour les deux).
- Reçu est composé par Transaction (1 à 1 : 1 -> 1).

Ce diagramme détaillé illustre la structure du système, montrant les différentes entités, leurs propriétés, leurs comportements et leurs interrelations.

5.2.3 Diagramme de séquence

Les flux ont été détaillés avec des diagrammes de séquence, qui modélisent le comportement dynamique du système en décrivant l'ordre chronologique des interactions entre les objets ou les acteurs pour accomplir une tâche spécifique. Ils sont essentiels pour comprendre les flux de données et les appels de méthodes.

Trois séquences clés ont été modélisées :

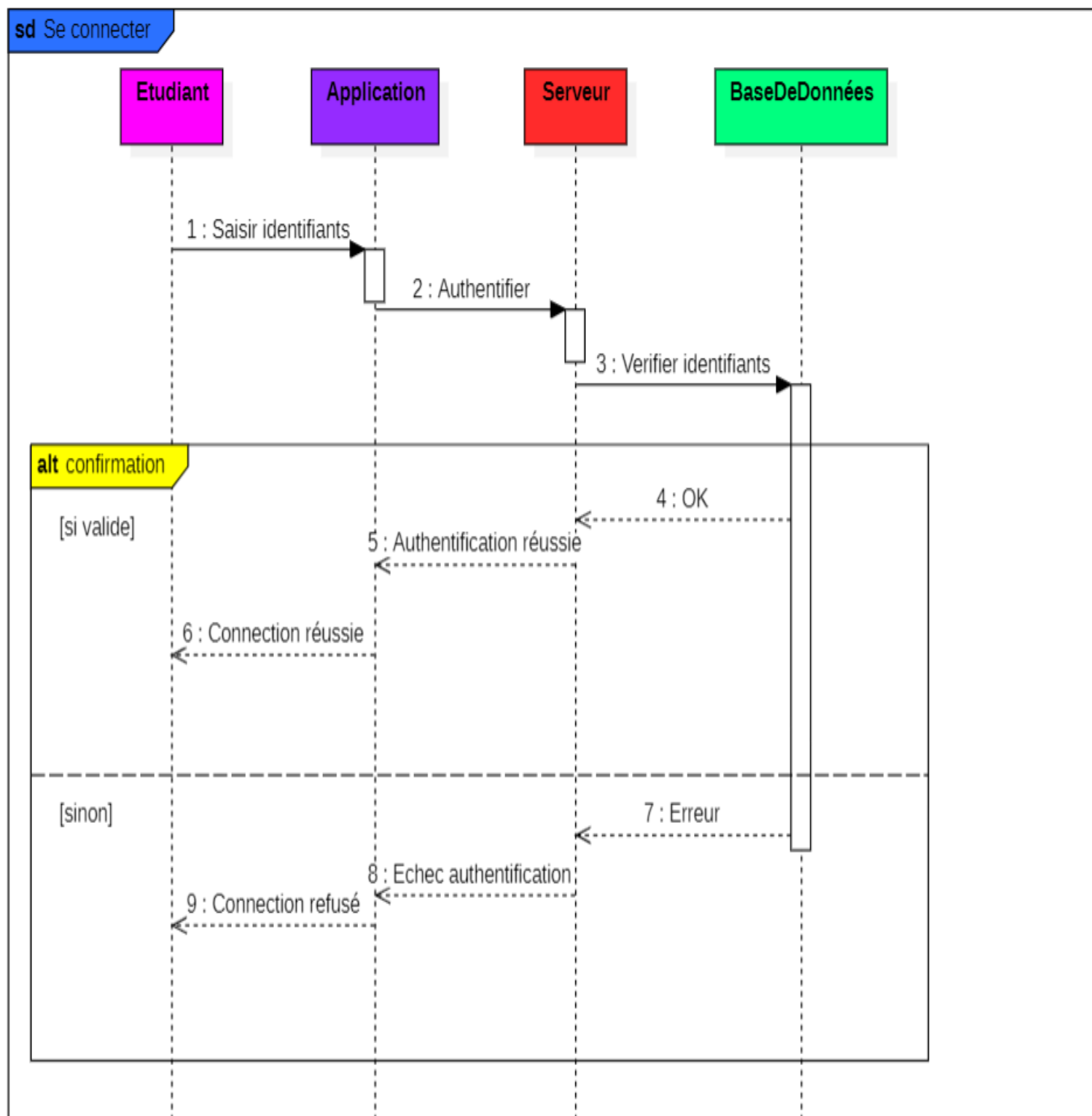


Figure 3 : Diagramme de Séquence : Se connecter

- **Séquence "Se connecter"** : Ce diagramme illustre l'échange de messages entre l'Étudiant, l'Application Mobile, le Serveur Backend (Django REST Framework) et la Base de données pour valider une connexion. Il détaille les étapes de soumission des identifiants, la requête au serveur, la vérification en base de données, la génération et le retour des tokens JWT, et le stockage de ces tokens côté client.

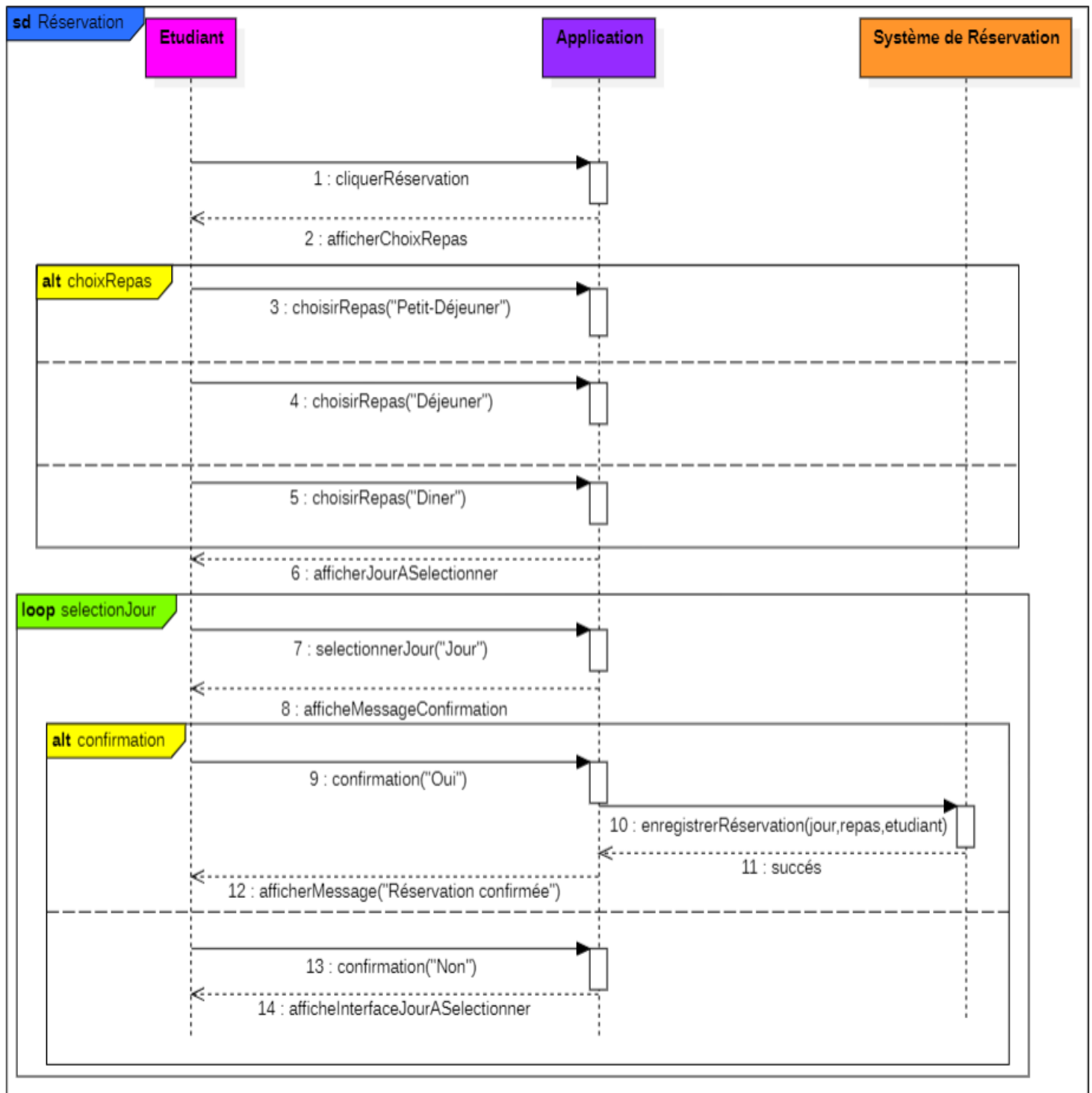


Figure 4 : Diagramme de Séquence : Réserveation

- **Séquence "Réserveation"** : Elle détaille les étapes depuis la sélection d'un repas et d'un jour par l'Étudiant sur l'Application Mobile, jusqu'à l'enregistrement de la réservation dans la base de données et l'affichage d'une confirmation à l'utilisateur. Cela inclut les vérifications de disponibilité et les interactions avec les modèles de Reservation, Jour et Periode.

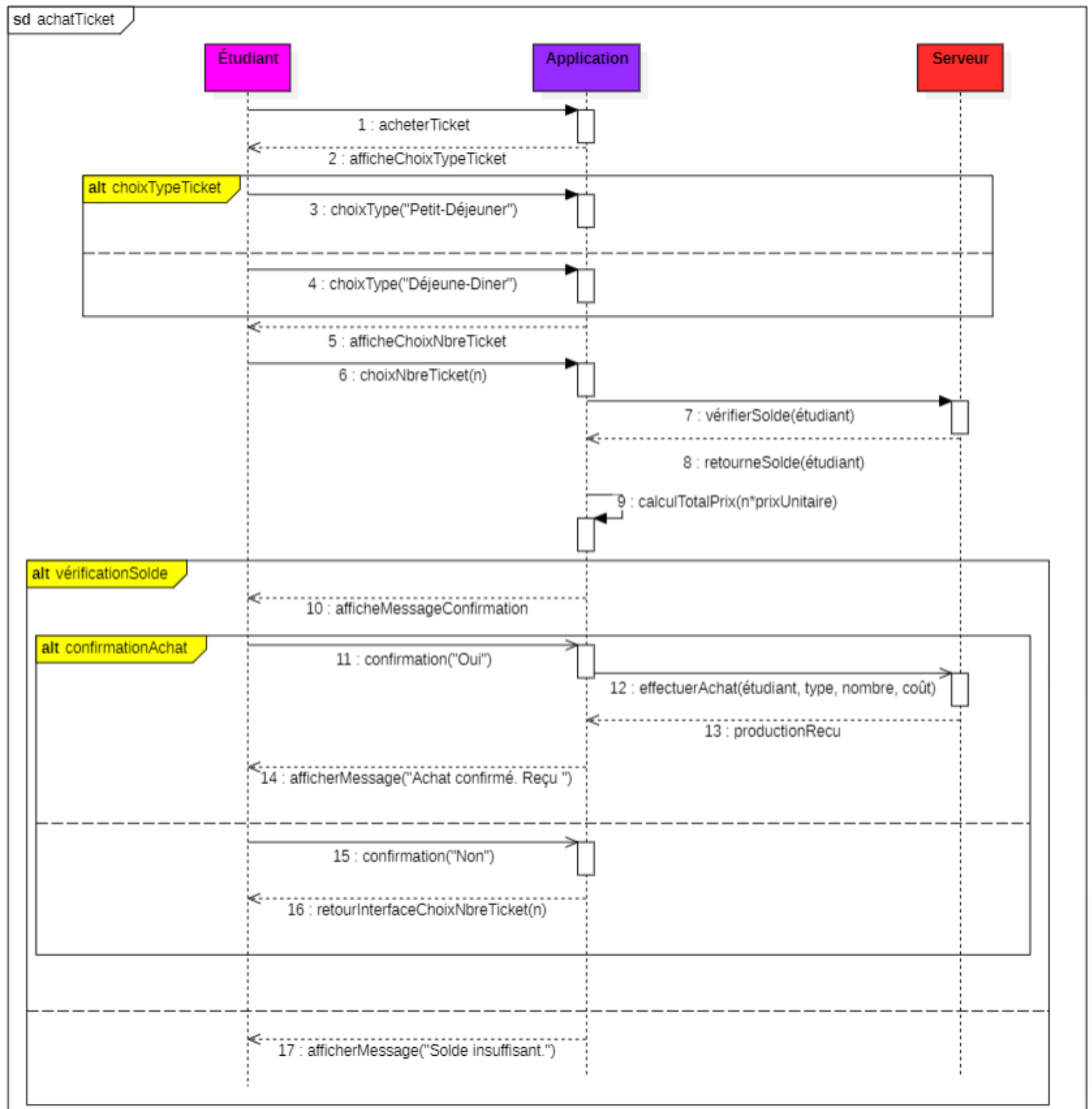


Figure 5 : Diagramme de Séquence : achatTicket

- **Séquence "Achat de tickets"** : Ce diagramme illustre le flux complet de la création d'une transaction de paiement. Il couvre la sélection de la quantité et

du type de ticket, l'appel à l'API de paiement, la génération d'un reçu, la mise à jour du solde de l'étudiant, et la création du ticket associé.

Ces diagrammes de séquence offrent une vision précise des interactions système, permettant d'anticiper les dépendances et de valider la logique des opérations complexes.

5.3 Conclusion du Chapitre 5

Cette phase de conception a constitué une étape déterminante dans la structuration de notre application de gestion du service restaurant de l'EMIG. Grâce à l'approche UML, nous avons pu clarifier, formaliser et anticiper les différentes interactions, processus métier et entités clés du système. Les diagrammes produits (cas d'utilisation, classes et séquences) ont permis de poser des bases solides avant le développement technique, en facilitant la compréhension des exigences fonctionnelles et non fonctionnelles du projet.

6. Réalisation de l'Application

6.1 Introduction

La phase de réalisation a constitué la partie la plus concrète du stage. Elle a permis de concrétiser les spécifications et les modèles définis lors de la conception, en traduisant l'architecture logicielle en un système fonctionnel. Cette étape a impliqué la mise en œuvre du backend, l'amélioration du frontend web, et la préparation de l'extension mobile, en s'appuyant sur les technologies sélectionnées.

6.2 Choix de l'environnement de développement

Un environnement de développement robuste a été choisi, guidé par les contraintes techniques et les besoins fonctionnels du projet, visant à garantir robustesse, performance et maintenabilité.

Pour ce projet, les technologies suivantes ont été privilégiées :

- **Éditeurs de code** : Des outils comme Android Studio, Visual Studio Code ont été utilisés. Ils fournissent des fonctionnalités d'assistance à la programmation, des émulateurs pour tester les applications sur différents appareils, et des systèmes d'intégration continue pour automatiser les processus de développement.

Pour le **backend (partagé entre l'application web et mobile)**, l'environnement de développement repose sur :

- **Framework** : Django, complété par Django REST Framework (DRF) pour la création d'API RESTful. Ce choix offre une structure solide et des outils puissants pour la gestion des données et la logique métier.
- **Base de données** : PostgreSQL est utilisé comme système de gestion de base de données, assurant une gestion fiable et performante des informations (encodage UTF8, collation fr_FR.UTF-8).
- **Authentification** : JWT (JSON Web Tokens) est implémenté via la bibliothèque SimpleJWT, fournissant un mécanisme d'authentification sécurisé et standardisé.

Pour le **frontend de l'application web**, l'environnement est configuré avec :

- **Framework** : React.js, choisi pour sa capacité à construire des interfaces utilisateur dynamiques et modulaires.
- **Styling** : TailwindCSS est utilisé pour une approche utilitaire et rapide du design.
- **Communication API** : Axios est la bibliothèque HTTP client privilégiée pour interagir avec le backend RESTful.

Pour le frontend de l'**application mobile** le choix technologique a été arrêté sur :

- **Framework** : React Native, avec une forte recommandation pour React Native en raison de sa capacité à générer des applications natives pour Android et iOS à partir d'un code unique, optimisant les efforts de développement.
- **Styling** : NativeCSS(similaire à TailwindCSS) est utilisé pour une approche utilitaire et rapide du design.

Enfin, pour la **génération de documents**, des bibliothèques comme ReportLab sont utilisées pour produire des reçus et rapports au format PDF.

Cet environnement de développement intégré a permis une collaboration efficace et une production de code de haute qualité pour l'ensemble de la plateforme

6.2.1 Architecture de l'application

L'architecture de la plateforme de gestion des services du restaurant a été pensée pour être **robuste, modulaire et évolutive**, en suivant une approche **client-serveur** reposant sur une **API RESTful**. Cette API constitue le point central d'échange entre les différentes interfaces (web et mobile) et le cœur du système.

La plateforme repose sur **deux principales composantes techniques** :

6.2.1.1 Backend (Serveur)

Le **backend** est développé en **Python** à l'aide des frameworks **Django** et **Django REST Framework (DRF)**. Il est responsable de la **logique métier**, de la gestion des données et de la fourniture des points d'accès (endpoints) pour les opérations réalisées par les utilisateurs.

6.2.1.1.1 Organisation du backend

Le code backend est organisé de manière modulaire :

- **models/** → définit les entités principales du système (Étudiant, Réservation, Ticket, Paiement, etc.) ;
- **serializers/** → assure la conversion des données entre les objets Python et le format JSON utilisé par l'API ;
- **views/** → gère la logique des API à l'aide des ViewSets de DRF ;
- **urls.py** → regroupe les routes API accessibles depuis le web ou le mobile.

6.2.1.2 Authentification et sécurité

L'accès au système est sécurisé par **JWT (JSON Web Tokens)** via le module **SimpleJWT**, qui permet une gestion sécurisée des sessions utilisateurs, avec des mécanismes d'expiration et de rafraîchissement des tokens.

6.2.1.3 Endpoints principaux

Les principales fonctionnalités exposées par l'API sont accessibles via des URL spécifiques :

- /api/etudiants/
- /api/reservations/
- /api/tickets/
- /api/paiements/
- /api/recharges/
- /api/jours/, /api/periodes/
- Endpoints d'authentification (/api/token/, /api/token/refresh/, /api/auth/register/, etc.)

6.2.1.4 Frontend Web (interface administrateur)

L'interface web est conçue sous la forme d'une **application monopage (SPA)** développée avec **React.js**.

Organisation

- **context/** → gestion de l'état utilisateur (authentifié ou non) ;
- **services/** → gestion des appels API et stockage sécurisé des tokens ;
- **pages/** → vues principales (connexion, gestion des réservations, tickets, etc.) ;
- **components/** → éléments réutilisables d'interface.

Le frontend web est principalement destiné aux administrateurs et responsables, tandis que l'architecture API permet l'intégration naturelle d'extensions mobiles.

6.2.1.5 Application mobile (extension pour les étudiants)

L'application mobile est développée avec **React Native**, garantissant une compatibilité multiplateforme (Android/iOS).

- **Fonctionnalités intégrées**
 - Authentification sécurisée
 - Réservations de repas
 - Historique des réservations
 - Génération de QR codes pour valider les repas

L'application communique avec le même **backend** que l'interface web grâce à des requêtes HTTP sécurisées.

6.2.2 Schéma de la base de données

Le système repose sur une base de données relationnelle **PostgreSQL**, reconnue pour sa robustesse et sa fiabilité.

Les principales entités modélisées sont :

Modèle	Fonction principale
Utilisateur	Base commune pour tous les types d'utilisateurs (étudiants, vendeurs, administrateurs)
Étudiant	Étend "Utilisateur" → ajout de champs spécifiques (matricule, solde, etc.)
Jour et Période	Organisation des repas par jour et par moment (Petit déjeuner, Déjeuner, Dîner)
Réservation	Association entre un étudiant, un jour et une période pour réserver un repas
Ticket	Enregistrement des tickets émis lors de la réservation
Paieement	Historique des transactions financières (solde ou espèces)
Notification	Messages envoyés aux utilisateurs (confirmation, rappel, etc.)

L'ensemble des données est interconnecté par des relations de clés étrangères, garantissant **cohérence et intégrité**. Un soin particulier est apporté à l'optimisation des performances par des indexations appropriées.

Pourquoi cette architecture ?

Cette organisation permet :

- **Clarté** : chaque composant a un rôle précis ;
- **Modularité** : le backend peut évoluer sans affecter l'interface web ou mobile ;
- **Réutilisabilité** : une même API alimente plusieurs interfaces (web et mobile) ;
- **Évolutivité** : le système est préparé pour intégrer de futures fonctionnalités comme les statistiques avancées ou la gestion complète des stocks.

6.2.3 Interface de l'application mobile (étudiants)

L'application mobile est conçue pour offrir aux étudiants une expérience simple, fluide et intuitive. Elle propose les fonctionnalités suivantes :

- **Page d'inscription**

Permet aux étudiants de créer un compte personnel en saisissant les informations nécessaires (nom, prénom, courriel, matricule, téléphone, mot de passe). Une fois inscrits, ils peuvent accéder à toutes les fonctionnalités de l'application.

- **Page de connexion**

Interface d'authentification permettant aux utilisateurs de se connecter à leur espace personnel à l'aide de leurs identifiants.



**Bienvenue sur
EMIGResto**

Numéro de matricule ou adresse mail

Mot de passe

Mot de passe oublié ?

SE CONNECTER

Vous n'avez pas de compte ? **Créer un compte**

Figure 6 : Page de connexion

Créer un nouveau compte

Nom

Prénom

Numéro de matricule

Adresse mail

Numéro de téléphone

Sexe : ☐ Féminin ☐ Masculin

Mot de passe

Confirmer le mot de passe

CRÉER UN COMPTE

Figure 7 : Page d'inscription

- **Page principale**

Cette page regroupe l'ensemble des fonctionnalités accessibles aux étudiants :

- **Consultation des tickets disponibles ;**
- **Réservation de repas :** possibilité de réserver un repas pour soi ou pour un tiers (par exemple un camarade) ;
- **Consultation de l'historique** des réservations passées et à venir ;

- **Accès aux notifications** : rappels de réservation, confirmations, messages importants.

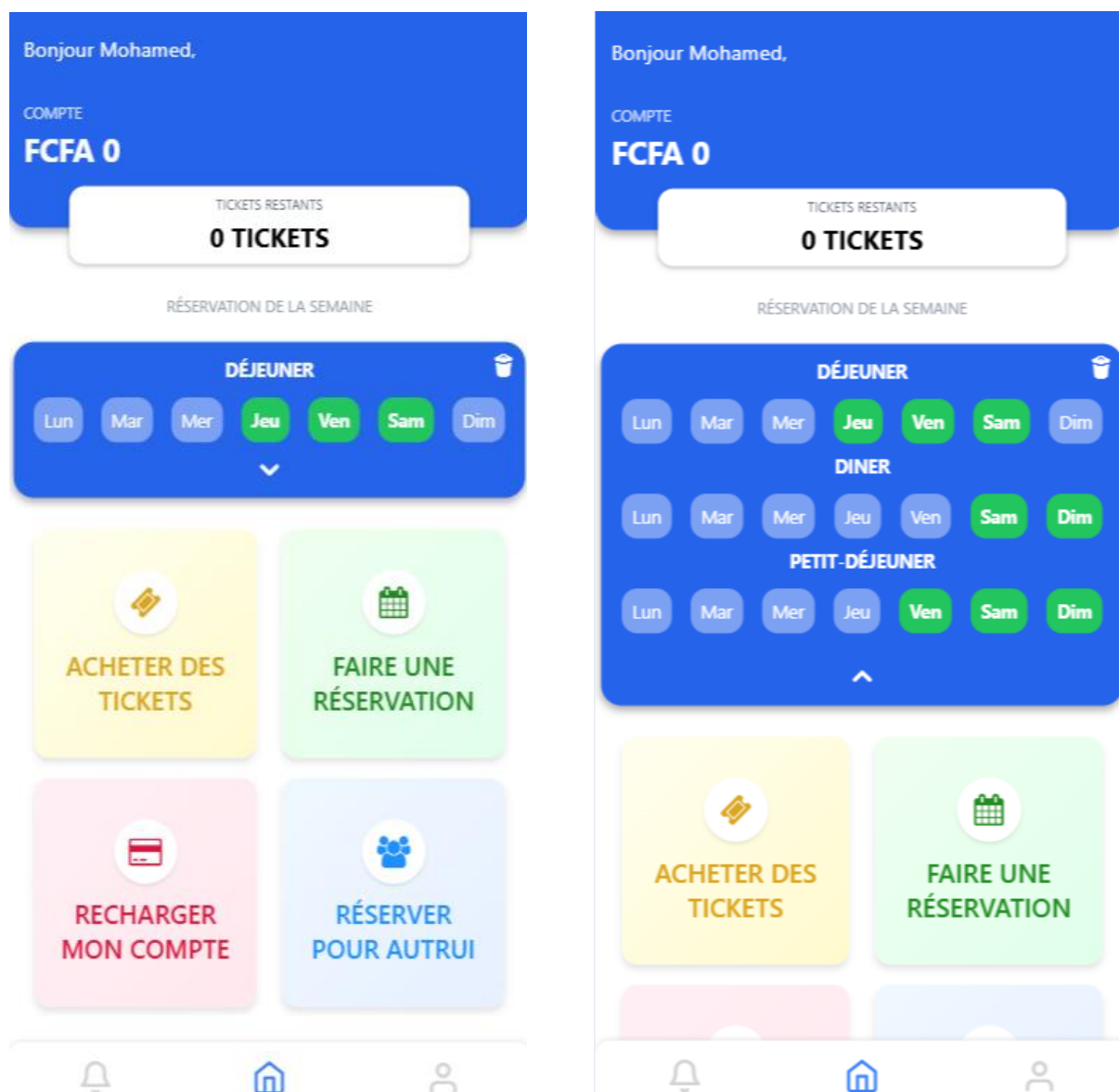


Figure 8 : Page principale

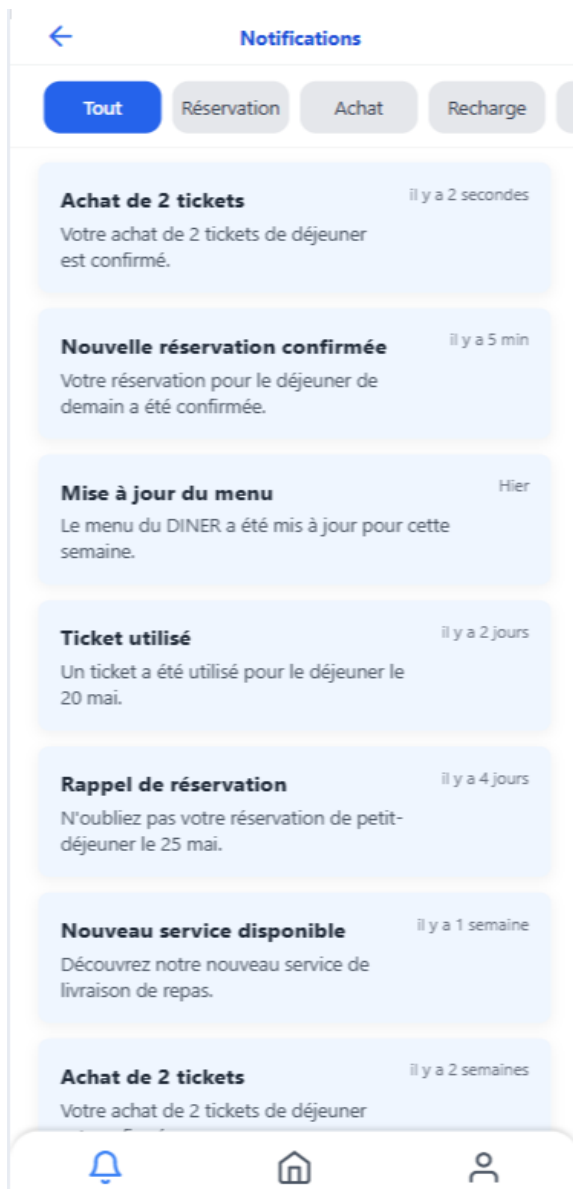


Figure 10 : Page de notifications

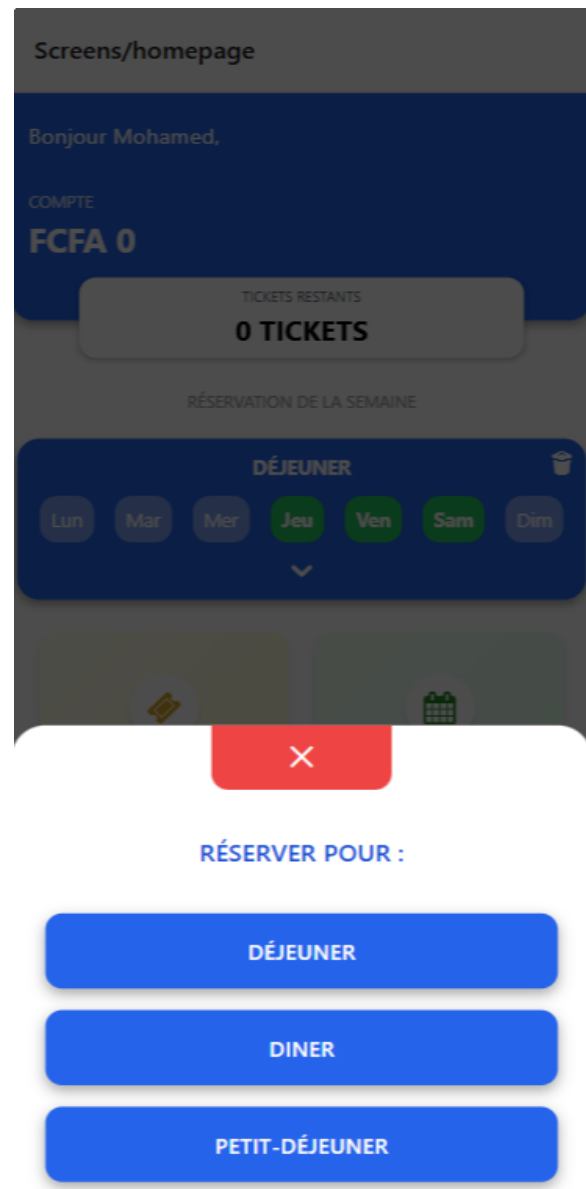


Figure 9 : Page principale : option réservation

NB : Certaines parties de l'interface ont été conçues en prévision de futures fonctionnalités, afin d'assurer la cohérence visuelle et l'évolutivité de l'application.

6.2.4 Connexion entre l'application mobile et le serveur

La communication entre l'application mobile ou l'application web frontend et le serveur backend repose entièrement sur une **API RESTful**. Cette approche standardisée garantit l'interopérabilité et la flexibilité, permettant à différents clients (web ou mobile) de consommer les mêmes services backend.

Le cœur de cette connexion est géré par la bibliothèque **Axios** côté frontend, configurée via le module `apiServices.js`. Ce module centralise la logique de communication et assure plusieurs fonctions cruciales :

- **Instance Axios** : Une instance d'Axios est créée avec une `baseURL` pointant vers l'API du backend.
- **Intercepteur de requêtes** : Avant chaque requête envoyée au backend, un intercepteur est utilisé pour injecter automatiquement le token d'accès JWT dans l'en-tête `Authorization: Bearer <token>`. Ce token est récupéré de l'espace de stockage local (`localStorage` via `tokenStorage.js`).
- **Intercepteur de réponses (gestion du 401)** : Un intercepteur est également configuré pour gérer les réponses du serveur. En cas de statut HTTP 401 (Unauthorized), indiquant un token d'accès expiré, l'intercepteur tente automatiquement de rafraîchir le token en utilisant le refresh token. Si le rafraîchissement est réussi, la requête initiale est rejouée avec le nouveau token. En cas d'échec du rafraîchissement, les tokens sont effacés, et l'utilisateur est potentiellement redirigé vers la page de connexion. Cette logique assure une expérience utilisateur transparente sans nécessiter de reconnexion manuelle fréquente.
- **Gestion centralisée des erreurs** : Toutes les erreurs d'API (réponses du serveur avec statut d'erreur) et les erreurs réseau sont capturées et traitées de manière centralisée, ce qui facilite le débogage et l'affichage de messages d'erreur cohérents à l'utilisateur.
- **Services génériques et spécifiques** : Une classe `BaseService` fournit des méthodes CRUD génériques (`list`, `get`, `create`, `update`, `delete`) pour interagir avec les endpoints de chaque entité métier (étudiants, réservations, tickets, etc.). Un

`AuthService` dédié gère les appels spécifiques à l'authentification (`login`, `register`, `me`, `logout`, `refresh`).

Cette architecture permet à l'application mobile de communiquer de manière sécurisée et efficace avec le backend, en exploitant les mêmes endpoints RESTful que l'application web, garantissant ainsi une cohérence des données et des fonctionnalités entre les différentes interfaces.

6.2.5 Tests et validation

Une stratégie de tests et de validation a été mise en place, essentielle pour garantir la qualité, la fiabilité et la robustesse de la plateforme de gestion des services du restaurant. Cette stratégie a couvert à la fois le backend et le frontend, ainsi que les scénarios de bout en bout.

Pour le **Backend (Django)**, l'approche de test repose sur pytest et coverage :

- **Tests unitaires** : Ils ciblent des fonctionnalités critiques telles que le hachage du mot de passe dans `UtilisateurSerializer.create()`, la contrainte `UniqueTogether` dans `ReservationSerializer` (pour éviter les doublons de réservation), et le comportement des méthodes `perform_create()` dans les `ViewSets` (par exemple, `TicketViewSet` qui fixe l'étudiant à l'utilisateur connecté).
- **Tests d'intégration** : Ils couvrent les principaux endpoints de l'API RESTful, y compris les endpoints d'authentification (`/api/auth/register/`, `/api/auth/token/`, `/api/auth/me/`, `/api/auth/token/refresh/`, `/api/auth/logout/`) et les opérations CRUD sur les entités clés comme `/api/etudiants/`, `/api/reservations/`, `/api/tickets/` [1, 1]. L'objectif est d'atteindre une couverture minimale de 80% du code.

Pour le **Frontend (React)**, les tests sont réalisés avec Jest et React Testing Library :

- **Tests unitaires** : Ils se concentrent sur des composants isolés et la logique, comme `AuthContext` (en simulant les appels API de login/logout), des composants simples comme `Spinner` ou `StudentSelect`, et en s'assurant que `PrivateRoute` redirige correctement les utilisateurs non authentifiés.
- **Tests d'intégration (basés sur les composants)** : Ils vérifient les interactions au sein des composants, par exemple, en simulant la saisie d'email/mot de passe sur `LoginPage` et en testant le succès ou le refus de connexion (avec des mocks d'API via `msw`), ou en vérifiant le calcul du prix total et la mutation de création de ticket dans `TicketForm`. Le

`ReservationGrid` est testé pour simuler les données à différentes étapes (chargement, affichage du tableau, interaction avec les cases à cocher).

Des **tests de bout en bout (E2E)** sont également envisagés avec Cypress ou Playwright pour valider les scénarios utilisateur clés :

- Scénarios complets comme l'inscription, la connexion et la navigation vers le tableau de bord.
- Le processus de vente de tickets (sélection d'étudiants, validation, vente réussie).
- La gestion des réservations (cocher/décocher des cases).

Cette approche de test multicouche permet de détecter les défauts à différents niveaux de l'application, assurant ainsi une plateforme fiable et stable.

6.3 Conclusion du Chapitre 6

Cette phase de réalisation a permis de concrétiser l'architecture et la conception de la plateforme de gestion des services du restaurant. Le choix judicieux des technologies (Django, React.js, PostgreSQL, JWT, avec une préconisation pour React Native pour le mobile) a permis de construire un système fonctionnel et modulaire. Les interfaces, qu'elles soient destinées aux étudiants ou au personnel administratif, ont été développées pour être intuitives et efficaces, couvrant des fonctionnalités clés telles que l'authentification, la réservation de repas, la gestion des stocks et la vente de tickets. La mise en place d'une communication robuste via une API RESTful et l'intégration de tests automatisés à chaque niveau du développement sont des gages de qualité et de fiabilité pour la solution déployée.

Conclusion Générale

• Résumé des résultats obtenus

Ce stage a constitué une expérience particulièrement enrichissante, tant sur le plan technique que personnel. Il a permis de contribuer activement à la conception et à la mise en place d'une plateforme numérique intégrée pour la gestion du service restauration de l'EMIG.

Le projet a abouti à la réalisation d'une application mobile fonctionnelle et à l'amélioration de l'infrastructure web existante, avec comme objectif principal de simplifier les démarches des étudiants et d'améliorer l'organisation interne du service.

Les principales fonctionnalités mises en place incluent :

- Un système d'authentification sécurisé et une gestion différenciée des utilisateurs ;
- La gestion numérique des tickets, des réservations et des paiements ;
- La génération de rapports automatisés pour une meilleure visibilité administrative ;
- Pour les étudiants : consultation des menus, réservation des repas, historique des réservations, paiement dématérialisé et notifications de rappel.

L'ensemble de ces fonctionnalités s'appuie sur une architecture robuste et évolutive, capable d'accompagner le développement futur du service.

• Apports du projet

Ce projet répond directement aux problématiques identifiées lors de l'analyse de l'ancien système manuel :

- Modernisation du service : digitalisation complète des processus de réservation et de gestion ;
- Gain d'efficacité : réduction des tâches répétitives et amélioration de la coordination entre les différents acteurs ;
- Meilleure expérience pour les utilisateurs : rapidité, accessibilité et transparence ;
- Fiabilité accrue : centralisation et traçabilité des opérations ;
- Préparation de l'avenir : une architecture conçue pour évoluer et accueillir de nouvelles fonctionnalités selon les besoins futurs de l'EMIG.

• Limites et perspectives

Comme tout projet en cours de construction, certains aspects restent à approfondir ou à optimiser. Les axes d'amélioration identifiés incluent notamment :

- Renforcement de la sécurité des échanges de données pour une utilisation en production ;
- Optimisation des performances pour garantir une meilleure réactivité à grande échelle ;
- Mise en place de tests automatisés avancés pour assurer une meilleure fiabilité technique ;
- Amélioration continue de l'expérience utilisateur, avec des fonctionnalités supplémentaires prévues pour les futures versions.

Ces perspectives constituent des pistes pour faire évoluer cette plateforme vers un niveau professionnel complet.

En résumé, ce travail a posé les bases solides d'une solution numérique moderne et cohérente, au service des étudiants et de l'administration de l'EMIG. Il représente une première étape importante dans la transformation numérique du service restauration, tout en ouvrant la voie à de futures améliorations telque mise en place d'un module d'IA pour la prédiction des réservations.

Bibliographie

Mahaman Lawan Abou Tasnim, Moussa Ismael projet industriel

Sitographie

www.postgresql.org

<https://tailwindcss.com>

<https://www.nativewind.dev/docs/getting-started/installation>

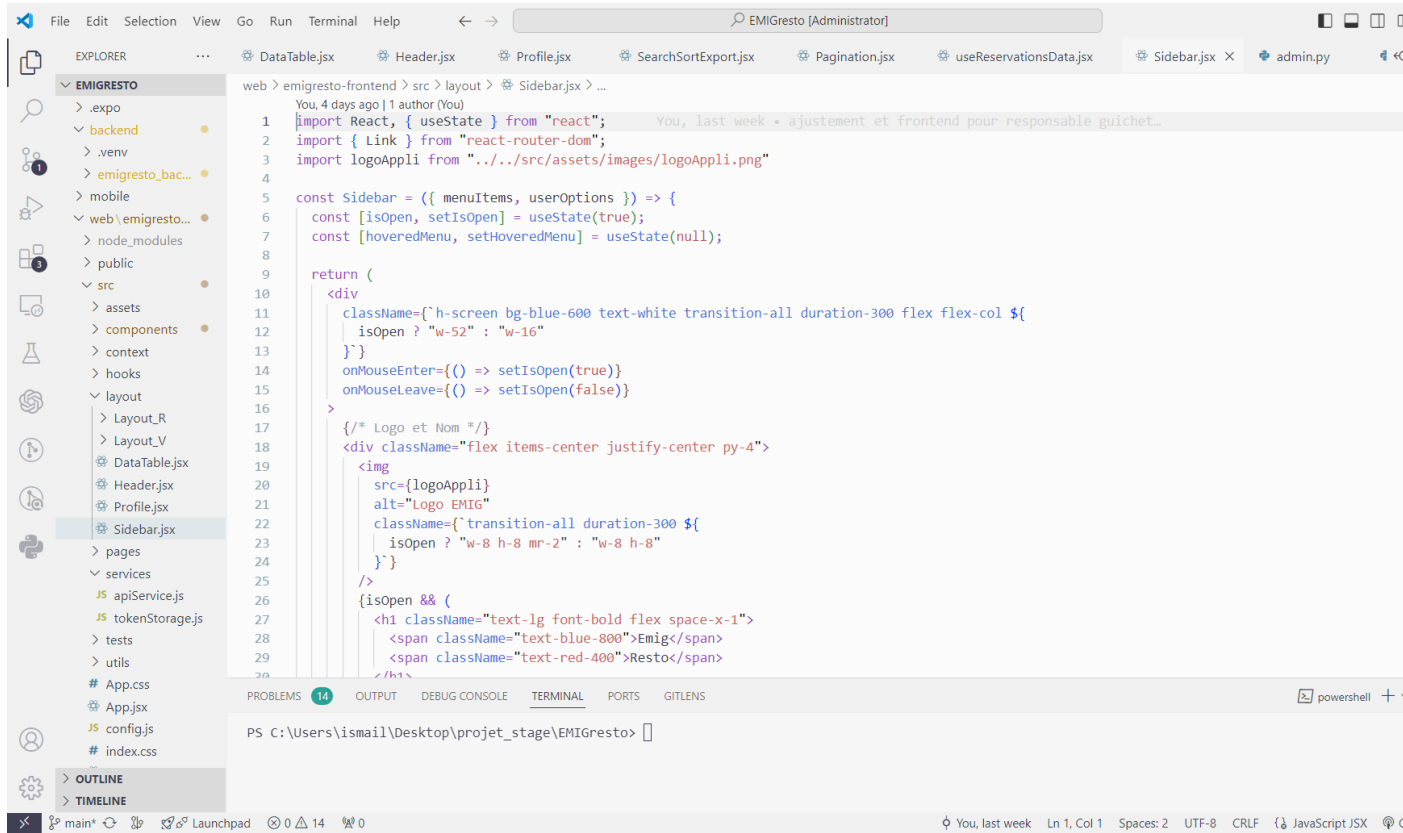
<https://www.nativewind.dev/docs/>

<https://www.djangoproject.com>

<https://www.codeur.com/blog/type-dapplication-mobile/>

Annexes

Annexe 1 : Portion du code source au frontend



```
File Edit Selection View Go Run Terminal Help
EMIGRESTO
> .expo
> backend
> .env
> emigresto_bac...
> mobile
> web \emigresto...
> node_modules
> public
> src
> assets
> components
> context
> hooks
> layout
> Layout_R
> Layout_V
> DataTable.jsx
> Header.jsx
> Profile.jsx
> Sidebar.jsx
> services
> apiService.js
> tokenStorage.js
> tests
> utils
> App.css
> App.jsx
> config.js
> index.css
> OUTLINE
> TIMELINE

web > emigresto-frontend > src > layout > Sidebar.jsx > ...
You, 4 days ago | 1 author (You)
You, last week • ajustement et frontend pour responsable guichet...

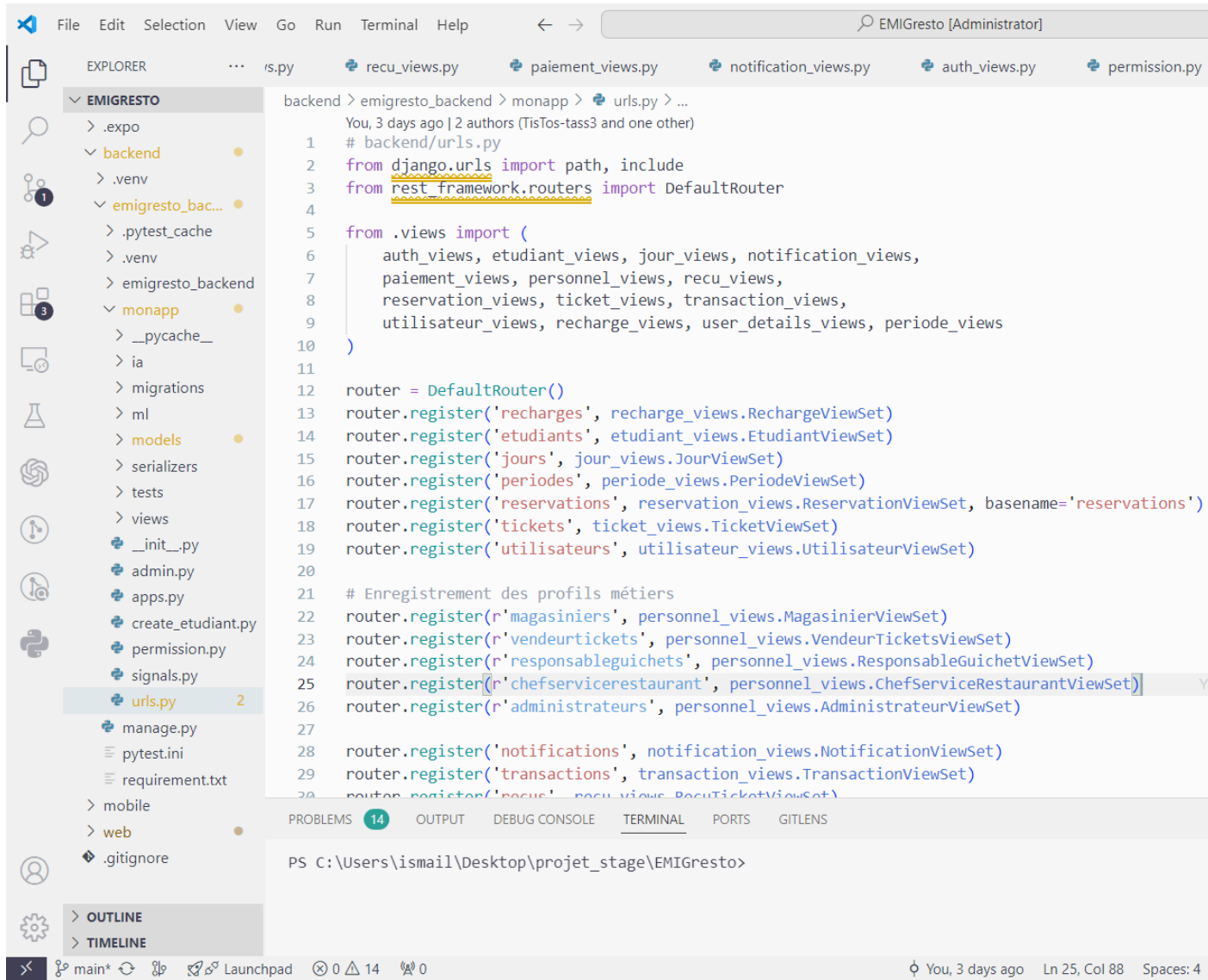
1 import React, { useState } from "react";
2 import { Link } from "react-router-dom";
3 import logoAppli from "../../src/assets/images/logoAppli.png"
4
5 const Sidebar = ({ menuItems, userOptions }) => {
6   const [isOpen, setIsOpen] = useState(true);
7   const [hoveredMenu, setHoveredMenu] = useState(null);
8
9   return (
10     <div
11       className={`h-screen bg-blue-600 text-white transition-all duration-300 flex flex-col ${
12         isOpen ? "w-52" : "w-16"
13       }`}
14       onMouseEnter={() => setIsOpen(true)}
15       onMouseLeave={() => setIsOpen(false)}
16     >
17       { /* Logo et Nom */ }
18       <div className="flex items-center justify-center py-4">
19         <img
20           src={logoAppli}
21           alt="Logo EMIG"
22           className={`transition-all duration-300 ${
23             isOpen ? "w-8 h-8 mr-2" : "w-8 h-8"
24           }`}
25         />
26         {isOpen && (
27           <h1 className="text-lg font-bold flex space-x-1">
28             <span className="text-blue-800">Emig</span>
29             <span className="text-red-400">Resto</span>
30           </h1>
31         )}
32       </div>
33     </div>
34   );
35 }
36
37 export default Sidebar;
```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS powershell +

PS C:\Users\ismail\Desktop\projet_stage\EMIGresto>

You, last week Ln 1, Col 1 Spaces: 2 UTF-8 CRLF JavaScript JSX

Annexe 2 : Portion de code au backend



The screenshot shows a Visual Studio Code editor with a Django project named 'EMIGRESTO'. The Explorer panel on the left displays the project structure, including folders like '.expo', 'backend', '.venv', 'emigresto_bac...', '.pytest_cache', 'emigresto_backend', 'monapp', and files like 'urls.py' (highlighted with a '2' icon). The main editor area shows the content of 'urls.py' in the 'monapp' folder. The code defines a 'DefaultRouter' and registers various API endpoints for different views. The Terminal panel at the bottom shows the command prompt 'PS C:\Users\ismail\Desktop\projet_stage\EMIGresto>'. The status bar at the bottom indicates the file is 'main*' and the cursor is at line 25, column 88.

```
File Edit Selection View Go Run Terminal Help
EMIGresto [Administrator]

EXPLORER
EMIGRESTO
  .expo
  backend
  .venv
  emigresto_bac...
  .pytest_cache
  .venv
  emigresto_backend
  monapp
    __pycache__
    ia
    migrations
    ml
    models
    serializers
    tests
    views
    __init__.py
    admin.py
    apps.py
    create_etudiant.py
    permission.py
    signals.py
    urls.py 2
    manage.py
    pytest.ini
    requirement.txt
  mobile
  web
  .gitignore
  OUTLINE
  TIMELINE

backend > emigresto_backend > monapp > urls.py > ...
You, 3 days ago | 2 authors (TisTos-tass3 and one other)
1 # backend/urls.py
2 from django.urls import path, include
3 from rest_framework.routers import DefaultRouter
4
5 from .views import (
6     auth_views, etudiant_views, jour_views, notification_views,
7     paiement_views, personnel_views, recu_views,
8     reservation_views, ticket_views, transaction_views,
9     utilisateur_views, recharge_views, user_details_views, periode_views
10 )
11
12 router = DefaultRouter()
13 router.register('recharges', recharge_views.RechargeViewSet)
14 router.register('etudiants', etudiant_views.EtudiantViewSet)
15 router.register('jours', jour_views.JourViewSet)
16 router.register('periodes', periode_views.PeriodeViewSet)
17 router.register('reservations', reservation_views.ReservationViewSet, basename='reservations')
18 router.register('tickets', ticket_views.TicketViewSet)
19 router.register('utilisateurs', utilisateur_views.UtilisateurViewSet)
20
21 # Enregistrement des profils métiers
22 router.register(r'magasiniers', personnel_views.MagasinierViewSet)
23 router.register(r'vendeurtickets', personnel_views.VendeurTicketsViewSet)
24 router.register(r'responsableguichets', personnel_views.ResponsableGuichetViewSet)
25 router.register(r'chefservicerestaurant', personnel_views.ChefServiceRestaurantViewSet)
26 router.register(r'administrateurs', personnel_views.AdministrateurViewSet)
27
28 router.register('notifications', notification_views.NotificationViewSet)
29 router.register('transactions', transaction_views.TransactionViewSet)
30 router.register('recus', recu_views.RecusTicketViewSet)

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\ismail\Desktop\projet_stage\EMIGresto>

main* Launchpad 0 14 You, 3 days ago Ln 25, Col 88 Spaces: 4
```

Annexe 3 : Interface du Django REST Framework

Django REST framework

Api Root / Produit List

Produit List

OPTIONS

GET

GET /api/produits/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

[]

Raw data

HTML

IdProduit

NomProduit

QuantiteDisponible

SeuilCritique

Ration

POST

Annexe 4 : Proposition de Digitalisation des Tickets de Réservation

Pendant notre stage, on a réfléchi à une amélioration du système actuel de gestion des tickets dans le restaurant de l'EMIG. On a eu l'idée de proposer un système de digitalisation des tickets, qui utiliserait des outils modernes comme la blockchain pour améliorer la sécurité et le suivi des ventes.

Le but de ce système serait de faire en sorte que chaque ticket soit traçable depuis sa création jusqu'à sa vente, et qu'on puisse éviter toute fraude ou ticket non autorisé. Dans notre idée, le service de comptabilité serait le seul à pouvoir générer des tickets, qui seraient ensuite attribués à un seul vendeur officiel, c'est-à-dire le magasinier.

Chaque lot de tickets serait créé avec un numéro unique (par exemple PKT-001-0001) et un type de repas (petit-déjeuner ou déjeuner/dîner). Les tickets seraient ensuite vendus uniquement via une application mobile sécurisée. L'application permettrait au vendeur de choisir un étudiant, de sélectionner le type de ticket et d'enregistrer la vente. Dès qu'un ticket est vendu, il est automatiquement lié à l'étudiant, ajouté à la base de données et un reçu est généré.

Grâce à ce système, la traçabilité est totale : chaque ticket est relié à un lot, à un vendeur, à un étudiant et à une date précise. Cela permettrait aussi de mieux suivre les paiements, les ventes quotidiennes et les stocks. On a pensé qu'on pourrait même utiliser une technologie inspirée de la blockchain pour que les tickets soient infalsifiables et enregistrés dans un système sécurisé.

Pour les étudiants, ce système serait aussi avantageux : ils pourraient consulter leurs tickets depuis l'application, voir l'historique de leurs repas, et réserver plus facilement. Cela rendrait le service plus pratique et plus rapide pour tout le monde.

Cette proposition n'a pas été développée techniquement pendant le stage, mais elle montre une piste sérieuse pour moderniser le fonctionnement du restaurant universitaire à l'avenir. Si elle est bien implémentée, elle pourrait résoudre plusieurs problèmes actuels et améliorer la gestion générale.